МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий фізико-технічний інститут

Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Проєктування високонавантажених систем»

# Лабораторна робота № 4. Реплікація MongoDB: відмовостійкість і лічильник

Виконав: Вовкотруб Олександр Віталійович
Група: ФБ-61мн

Перевірив: Родіонов А. М.

Київ 2026

# Зміст

**1 Мета роботи** **2**

**2 Постановка задачі** **2**

**3 Хід виконання** **2**

3.1 Середовище . . . 2
3.2 Розгортання replica set . . . 3
3.3 Частина I: write concern, read concern і перевибори . . . 4
3.3.1 Сценарій 1: `w: 3` без тайм-ауту . . . 4
3.3.2 Сценарій 2: `w: 3` з `wtimeout: 5000` . . . 5
3.3.3 Сценарій 3: перевибори primary . . . 7
3.4 Клієнт MongoDB і retryable writes . . . 9
3.5 Частина II: програма `mongo-bench` . . . 10
3.6 Частина III: веб-лічильник на MongoDB . . . 10
3.7 Методика вимірювань . . . 10
3.8 Аналітичний ноутбук . . . 11

**4 Результати** **11**

4.1 Частина I: що показали сценарії . . . 11
4.2 Частина II: час і точність лічильника . . . 12
4.3 Відмова primary: що стається з даними . . . 14
4.4 Частина III: порівняння сховищ веб-лічильника . . . 15

**5 Додаткове дослідження** **16**

5.1 Відкат і аномалії читання . . . 17
5.1.1 Rollback з rollback-файлами . . . 17
5.1.2 Аномалії читання . . . 25
5.1.3 Розподіл затримок і пауза при відмові . . . 27
5.2 Пропускна здатність . . . 28
5.2.1 Вузьке місце: синхронізація журналу . . . 29
5.2.2 Гарячий документ і лічильник з кількох документів . . . 31
5.2.3 Що не дало приросту . . . 32
5.3 Керована зміна primary і налаштування виборів . . . 33

**6 Висновки** **36**

**7 Джерела** **37**

# 1 Мета роботи

Розгорнути replica set MongoDB з трьох вузлів (primary і два secondary) і перевірити на ньому, як write concern і read concern визначають, коли запис вважається виконаним і що з ним стається при відмові вузла. Показати перевибори primary. Виміряти швидкість і точність конкурентного лічильника на `findOneAndUpdate` з `$inc` при `w: 1` і `w: "majority"`, зокрема з раптовою відмовою primary посеред навантаження. Використати MongoDB як сховище веб-лічильника з лабораторної роботи № 1 і порівняти його з пам’яттю, диском, PostgreSQL і Hazelcast.

# 2 Постановка задачі

Робота складається з трьох частин.

Частина I. Налаштувати реплікацію в конфігурації P-S-S (вузли можуть бути окремими процесами) [1…3] і перевірити:

1. запис з `writeConcern: { w: 3 }` без тайм-ауту, коли один вузол вимкнено; поки запис чекає, увімкнути вузол;
2. те саме зі скінченним `wtimeout`: дочекатися тайм-ауту, перевірити, чи дані записались і чи читаються з `readConcern: "majority"`;
3. перевибори primary після вимкнення поточного primary [4] і те, що після повернення старий primary отримує дані, записані без нього.

Частина II. Колекція з лічильником лайків. 10 окремих клієнтів одночасно збільшують його по 10 000 разів через `findOneAndUpdate` з `$inc` [5], тобто очікується 100 000. Чотири прогони: `w: 1`; `w: "majority"`; `w: 1` з вимкненням primary під час роботи; `w: "majority"` з вимкненням primary. Для кожного виміряти час і перевірити фінальне значення. Рядок підключення має перелічувати всі вузли: `mongodb://h1,h2,h3/?replicaSet=...`.

Частина III. Використати кластер MongoDB як сховище веб-лічильника з роботи № 1 (`findOneAndUpdate`, `w: "majority"`), навантажити його 10 клієнтами по 10 000 запитів і звести в таблицю з пам’яттю, диском, PostgreSQL і Hazelcast (загальний час, пропускна здатність).

Протокол: команди та їхні результати у вигляді знімків екрана, і аналіз.

# 3 Хід виконання

## 3.1 Середовище

Три процеси `mongod`, клієнти `mongo-bench`, `counter-server` і `loadgen` працюють на одній машині й спілкуються через loopback. Дані всіх трьох вузлів лежать на одному SSD, тому кожен вузол платить за `fsync` той самий диск. Контейнери не використовувались.

Таблиця 3.1. Апаратне і програмне середовище

| Параметр | Значення |
|---|---|
| CPU | Intel Coffee Lake (інженерний зразок, 2,60 ГГц), 8 ядер, 16 потоків, 0,8…4,4 ГГц, L3 16 МіБ |
| RAM | 62 ГіБ |
| Диск | NVMe SSD Samsung 970 EVO Plus 1 TB, ext4, розділ заповнений приблизно на 98 % |
| ОС | Fedora Linux 44, ядро 7.2.11-200.fc44.x86_64 |
| Мова | Swift 6.4, збірка `-c release` |
| СКБД | MongoDB Community 8.2.12 і mongosh 2.10.0 з Nix flake |
| Драйвер | MongoKitten 7.16.3, retryable writes реалізовано в роботі |
| Replica set | `rs0`, 3 вузли `127.0.0.1:21430...21432`, однаковий `priority: 1`, `electionTimeoutMillis: 10000` |
| WiredTiger | кеш 0,25 ГБ на вузол, журнал увімкнено (за замовчуванням) |

## 3.2 Розгортання replica set

Скрипт `scripts/mongo.sh` керує трьома вузлами. Кожен вузол запускається окремим процесом зі своїм каталогом даних і логом:

```
mongod --replSet rs0 --port 21430 --bind_ip 127.0.0.1 \
    --dbpath .mongodata/node1 --logpath reports/logs/mongo-node1.log \
    --wiredTigerCacheSizeGB 0.25 --quiet --timeStampFormat iso8601-utc \
    --fork
```

При першому запуску скрипт ініціалізує набір з трьох членів [3]:

```
rs.initiate({ _id: "rs0", members: [
    { _id: 0, host: "127.0.0.1:21430" },
    { _id: 1, host: "127.0.0.1:21431" },
    { _id: 2, host: "127.0.0.1:21432" } ] })
```

Клієнти підключаються за рядком, що перелічує всі вузли:

```
mongodb://127.0.0.1:21430,127.0.0.1:21431,127.0.0.1:21432/?replicaSet=rs0
```

Решта команд скрипта: `stop-node N` (SIGTERM, штатна зупинка), `kill-node N` (SIGKILL, раптова відмова), `start-node N`, `status` (стан вузлів з погляду primary), `wait-primary`, `wait-down N` (primary бачить вузол N недоступним), `wait-secondary N` (вузол N став SECONDARY і наздогнав primary) і `shell [N]` (mongosh до набору або напряму до вузла).

## 3.3 Частина I: write concern, read concern і перевибори

Скрипт `scripts/task4-part1.sh` виконує три сценарії і записує протокол кожного у `reports/logs/task4-part1-{1,2,3}.txt`. Команди оболонки мають мітку часу UTC. Сесії mongosh отримують команди зі stdin, а скрипт ставить кожну введену команду після підказки, яку mongosh для неї надрукував, тому протокол виглядає як інтерактивна сесія. Рисунки 3.1…3.8 є знімками цих протоколів у вигляді вікна терміналу, кожен поділено на дві або три частини.

Після SIGTERM процес mongod 8 виходить лише через 15,0 с. У лозі вузла в цей момент є `Entering quiesce mode for shutdown з quiesceTimeMillis: 15000`. Роль primary передається одразу (сценарій 3).

### 3.3.1 Сценарій 1: `w: 3` без тайм-ауту

Вузол 3 зупинено, primary позначив його недоступним. У терміналі 2 вставка з `{ w: 3, wtimeout: 0 }` чекає, доки документ запишуть усі три вузли (рис. 3.1). Тим часом у терміналі 1 документ уже читається на primary, зокрема з `readConcern("majority")`, бо його мають 2 вузли з 3. Після `start-node 3` вставка повертає `acknowledged: true` (рис. 3.2).

```
task4-part1-1.txt (1/2)

# Сценарій 1. Один secondary вимкнено, запис з writeConcern { w: 3, wtimeout: 0 } (без тайм-ауту).
# Час у [квадратних дужках] і в ISODate(...) указано в UTC.
# mongosh читає команди зі stdin: текст після підказки rs0 [...]> і є введеною командою.

[17:45:51.038] $ scripts/mongo.sh status
rs0, term 1, seen from node1
node1  127.0.0.1:21430  PRIMARY                 health 1  optime 1793641547:16  (pid 553720)
node2  127.0.0.1:21431  SECONDARY               health 1  optime 1793641547:16  lag 0 s  sync node1  (pid 553814)
node3  127.0.0.1:21432  SECONDARY               health 1  optime 1793641547:16  lag 0 s  sync node1  (pid 553903)

[17:45:53.010] $ scripts/mongo.sh stop-node 3
node 3 stopped (SIGTERM, pid 553903), exited after 15.0 s
[17:46:08.196] $ scripts/mongo.sh wait-down 3
node1 sees node3 as (not reachable/healthy) after 0.5 s (Error connecting to 127.0.0.1:21432 :: caused by :: onInvoke
:: caused by :: Connection refused)
[17:46:09.497] $ scripts/mongo.sh status
rs0, term 1, seen from node1
node1  127.0.0.1:21430  PRIMARY                 health 1  optime 1793641567:1  (pid 553720)
node2  127.0.0.1:21431  SECONDARY               health 1  optime 1793641567:1  lag 0 s  sync node1  (pid 553814)
node3  127.0.0.1:21432  (not reachable/healthy)  health 0  optime 0:0  (process stopped)

# [термінал 2] вставка з w: 3 чекає, доки документ запишуть усі три вузли
[17:46:10.531] $ scripts/mongo.sh shell --quiet --norc
rs0 [primary] test> use lab4
switched to db lab4
rs0 [primary] lab4> let t0 = new Date(); t0
ISODate('2026-11-02T17:46:11.504Z')
rs0 [primary] lab4> db.part1.insertOne({ scenario: 1, note: "w: 3, wtimeout: 0" }, { writeConcern: { w: 3, wtimeout:
0 } })
```

Рисунок 3.1. Сценарій 1, частина 1: вузол 3 зупинено, вставка з `w: 3` без тайм-ауту почала чекати

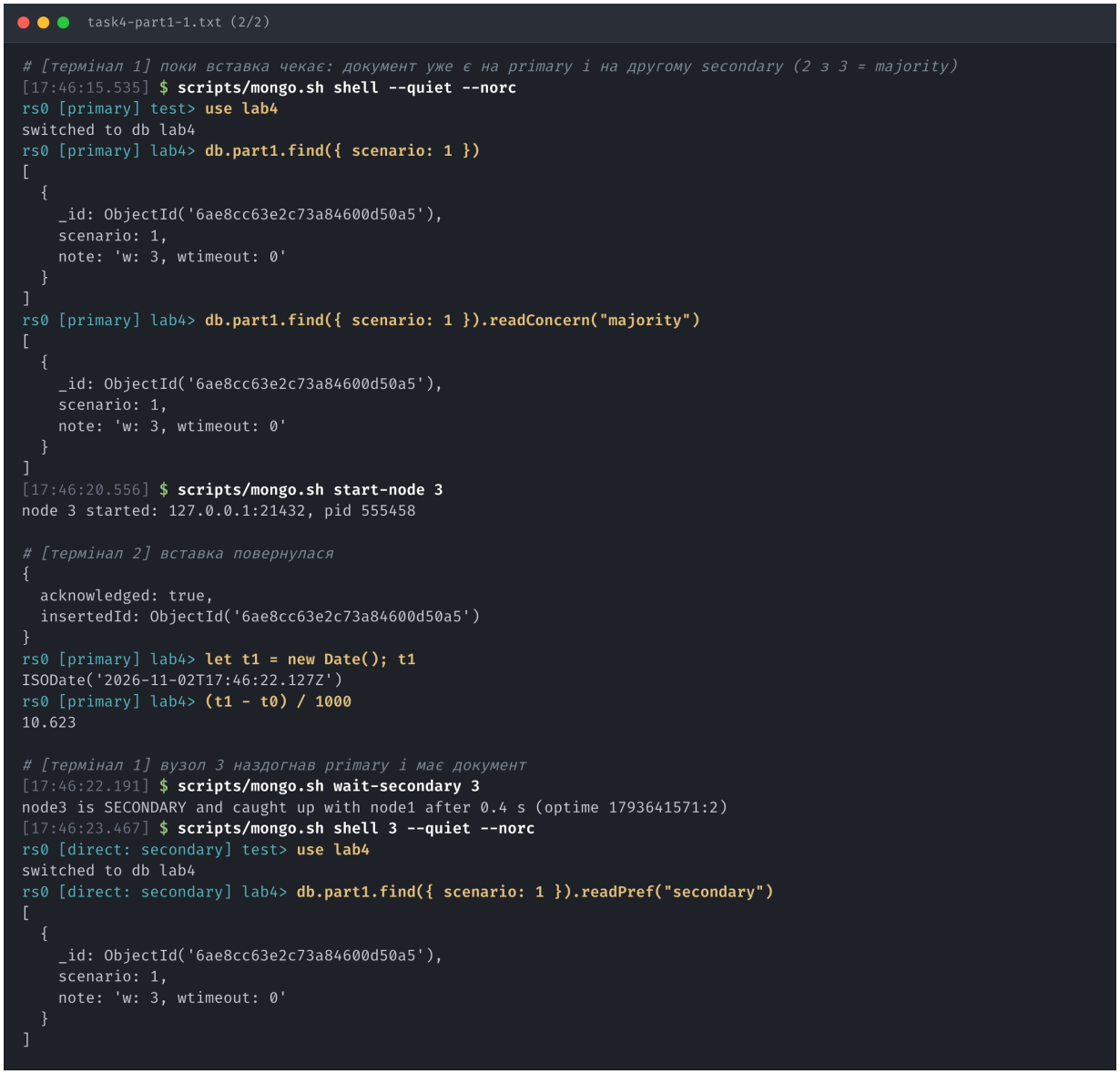



Рисунок 3.2. Сценарій 1, частина 2: читання під час очікування, повернення вузла 3, відповідь вставки через 10,623 с і документ на вузлі 3

### 3.3.2 Сценарій 2: `w: 3` з `wtimeout: 5000`

Вузол 3 знову зупинено, і та сама вставка йде з `wtimeout: 5000` (рис. 3.3). Через 5,049 с вона завершується помилкою `WriteConcernTimeout` (код 64), але у відповіді `n: 1`, і документ читається на primary, зокрема з `readConcern("majority")` (рис. 3.4). Він є і на node2. Після повернення вузол 3 наздогнав primary за 1,8 с і має той самий документ (рис. 3.5).

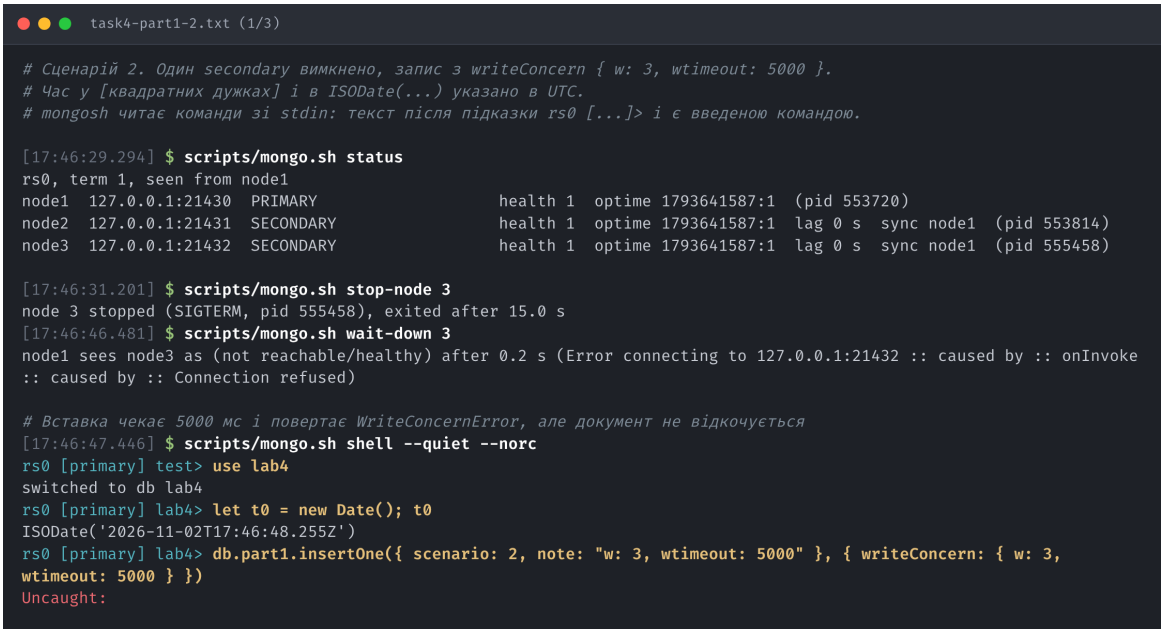



Рисунок 3.3. Сценарій 2, частина 1: зупинка вузла 3 і вставка з `w: 3, wtimeout: 5000`

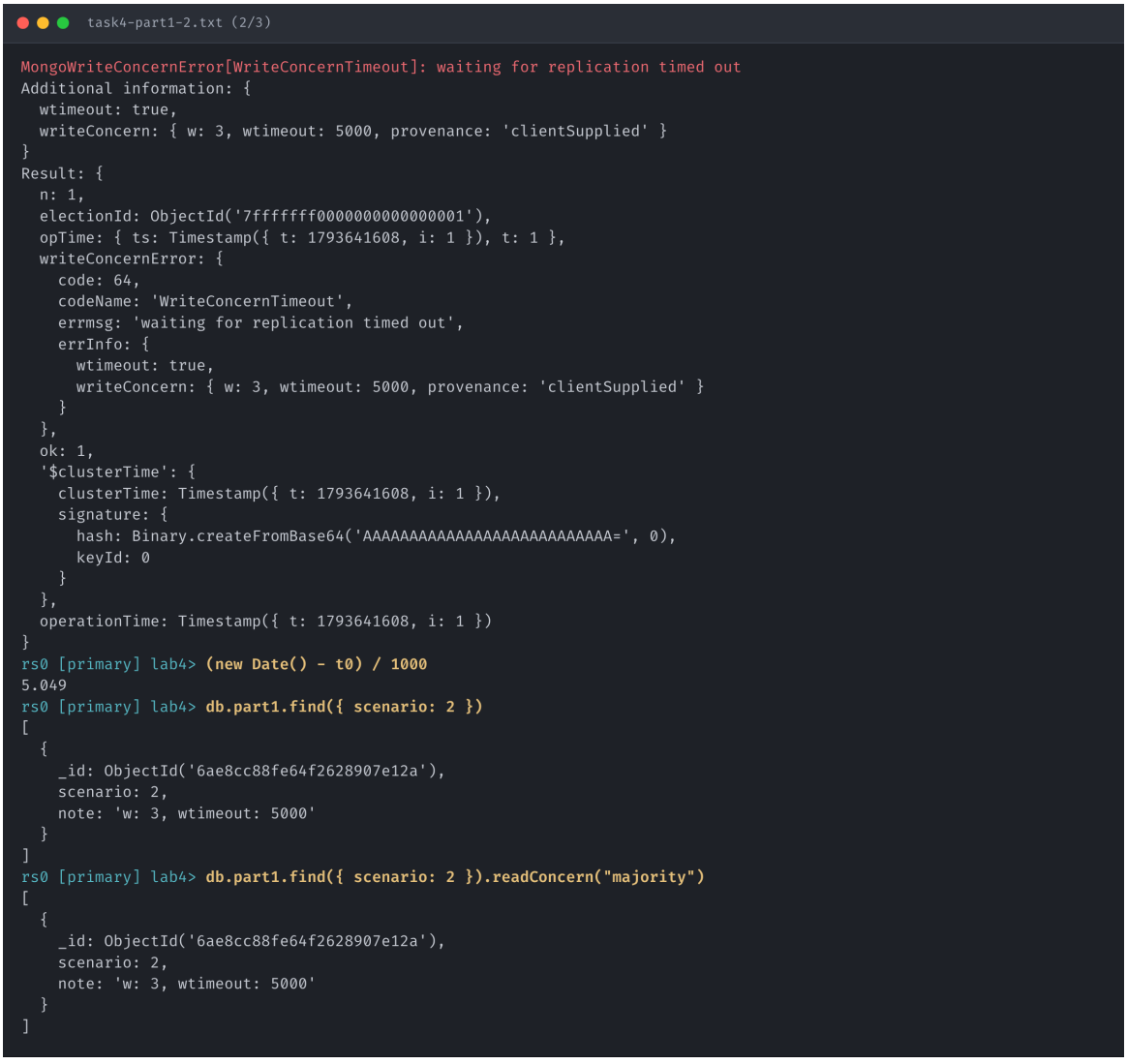



Рисунок 3.4. Сценарій 2, частина 2: через 5 с вставка повертає `WriteConcernTimeout` з `n: 1`, документ читається на primary з `majority`

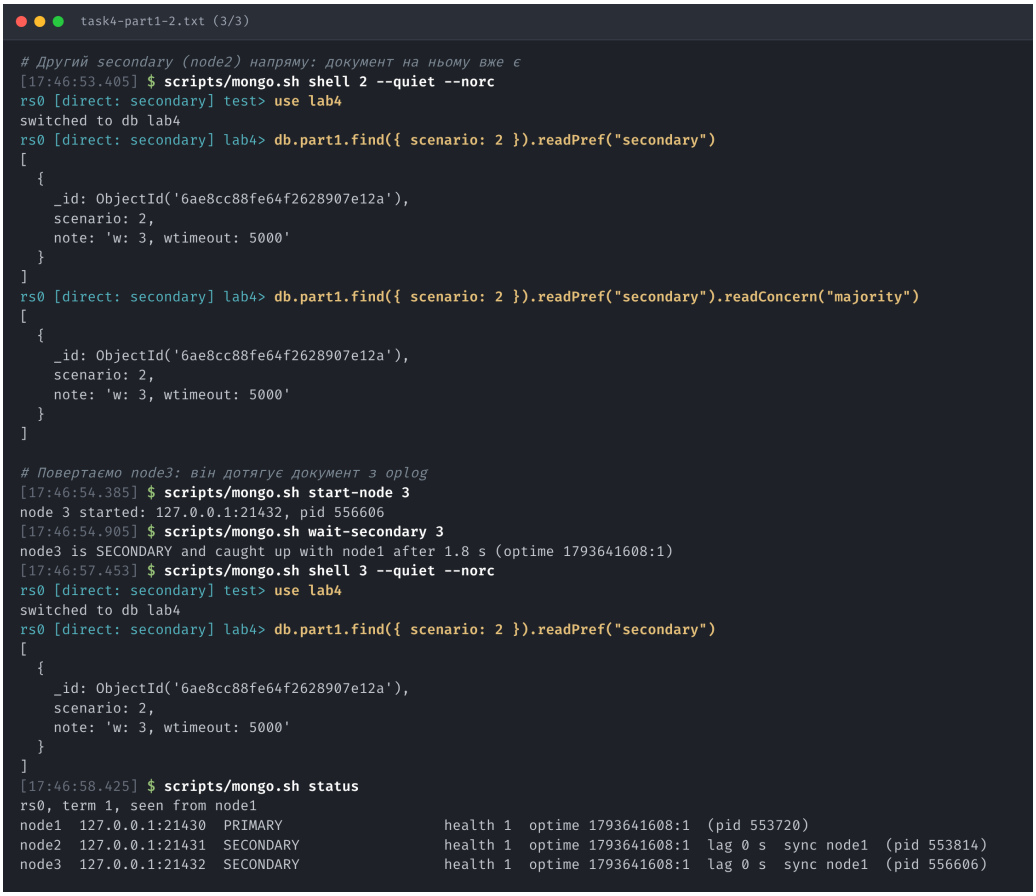



Рисунок 3.5. Сценарій 2, частина 3: документ на node2 з `majority` і на node3 після його повернення

### 3.3.3 Сценарій 3: перевибори primary

Поточний primary node1 штатно зупинено, новим primary став node2 у term 2 (рис. 3.6). Метрики виборів на node2 показують причину і час виборів, а через нього записано три документи з `w: "majority"` (рис. 3.7). Після повернення node1 приєднався як SECONDARY, наздогнав node2 за 1,7 с і має всі три документи (рис. 3.8).

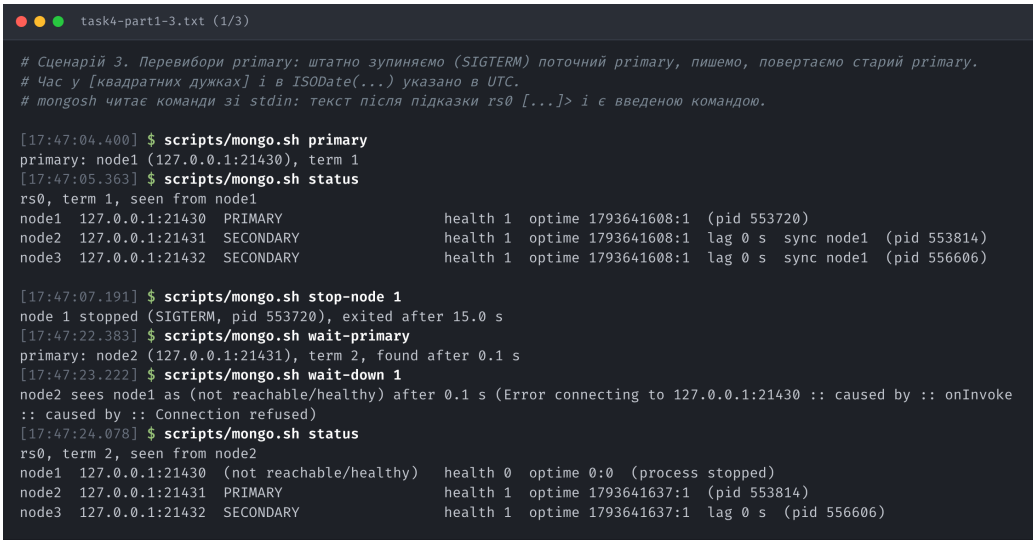



Рисунок 3.6. Сценарій 3, частина 1: зупинка primary node1, новий primary node2 у term 2

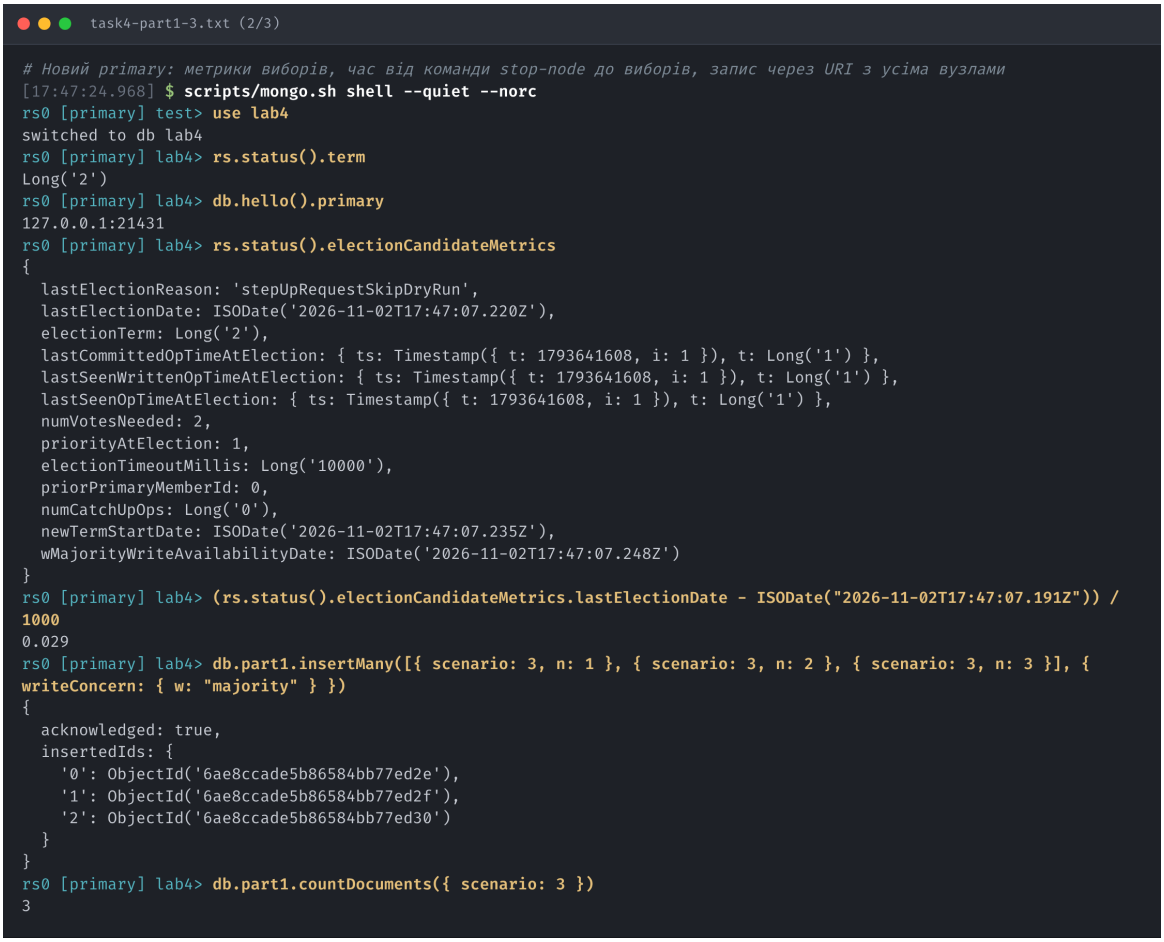



Рисунок 3.7. Сценарій 3, частина 2: метрики виборів на новому primary і запис трьох документів з `w: "majority"`

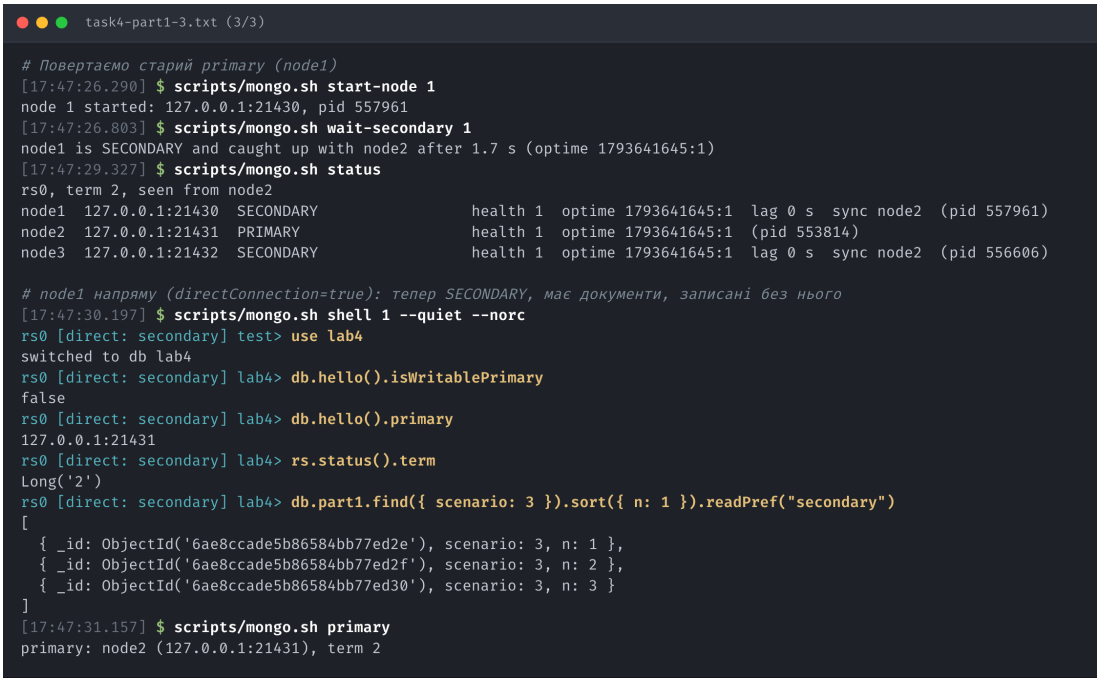



Рисунок 3.8. Сценарій 3, частина 3: старий primary повернувся як SECONDARY і має документи, записані без нього

## 3.4 Клієнт MongoDB і retryable writes

Для Swift використано драйвер MongoKitten [11]. Він знаходить primary за рядком підключення зі списком вузлів, але не реалізує retryable writes [8, 10]: його `findOneAndUpdate` не надсилає `txnNumber`, а сам драйвер нічого не повторює. Без них інкремент, відповідь на який загубилася разом з primary, не можна безпечно повторити: він міг бути вже застосований.

Тому клієнт `MongoCounterClient` (`Sources/CounterCore/`) сам будує `findAndModify`, як офіційні драйвери: власна логічна сесія з випадковим UUID (`lsid`) і наступний `txnNumber` на кожен логічний запис:

```
var command: Document = [
    "findAndModify": location.collection,
    "query": ["_id": documentID ?? location.documentID] as Document,
    "update": update,                  // ["$inc": ["likes": 1]]
    "new": true,
    "upsert": true,
    "writeConcern": (writeConcern ?? self.writeConcern).document,
]
if retryableWrites {
    command["lsid"] = ["id": sessionID] as Document
    command["txnNumber"] = lastTransactionNumber
        .add(1, ordering: .relaxed).newValue
}
```

Повтор надсилає ту саму команду з тим самим `txnNumber`. Сервер пам’ятає останній виконаний `txnNumber` кожної сесії (у `config.transactions`, що реплікується разом із самим записом) і на повтор уже виконаного запису повертає збережений результат. Повторюються лише тимчасові помилки за специфікацією [10], з паузою від 10 до 250 мс і лімітом 60 с:

```
while true {
    do {
        return MongoOutcome(value: try await operation(), retries: retries)
    } catch {
        guard MongoCommandFailure.isRetryable(error), clock.now < giveUp
        else { throw error }
        retries += 1
        if let failure = error as? MongoCommandFailure,
           failure.isNotPrimary { try? await cluster.reconnect() }
        try await Task.sleep(for: backoff)
        backoff = min(backoff * 2, Self.maximumBackoff)
    }
}
```

`reconnect()` потрібен, коли вузол відповів, що він більше не primary: MongoKitten оновлює відомості про вузли лише раз на 10 с. Читання `value()` іде з `readConcern: "majority"`, тож не бачить записів, які ще можуть бути відкочені.

## 3.5 Частина II: програма `mongo-bench`

`mongo-bench` (`Sources/mongo-bench/`) скидає лічильник у 0 з `w: "majority"`, створює 10 окремих клієнтів (у кожного свої з'єднання і своя сесія) і в `withThrowingTaskGroup` запускає по 10 000 інкрементів на клієнта. `ContinuousClock` вимірює лише фазу інкрементів.

З параметром `--kill-primary-after T` програма через T секунд знаходить primary командою `hello` і вбиває його через `scripts/mongo.sh kill-node` (SIGKILL, тобто раптова відмова без передачі ролі). Коли всі клієнти закінчили, вузол запускається знову, програма чекає, доки він стане SECONDARY і наздожене primary, робить один запис з `w: "majority"` в окремий документ, щоб усе попереднє стало majority-committed, і читає фінальне значення з `readConcern: "majority"`. Втраченими вважаються підтверджені сервером інкременти, яких немає у фінальному значенні.

Контрольний прогін `--no-retryable-writes` повторює інкременти після помилок так само, але без `lsid` і `txnNumber`.

## 3.6 Частина III: веб-лічильник на MongoDB

Сховище `MongoCounterStore` реалізує той самий протокол `CounterStore`, що й сховища робіт №1…3. `GET /inc` виконує описаний вище `findAndModify` з `$inc` і `w: "majority"`, `GET /count` читає значення з `readConcern: "majority"`. mongod обробляє запити одного з'єднання по черзі, а `txnNumber` однієї сесії має лише зростати, тому сховище тримає пул з 10 клієнтів і позичає кожному HTTP-запиту один з них:

```
public func increment() async throws {
    try await clients.withLease { _ = try await $0.increment() }
}
```

Сервер запускається так:

```
counter-server --store mongo --mongo-clients 10 --mongo-url "$MONGO_URL"
```

де `MONGO_URL` є наведеним вище рядком підключення з усіма трьома вузлами.

## 3.7 Методика вимірювань

1. `scripts/task4-part1.sh` виконав три сценарії частини I 02.11.2026 з 17:45 до 17:47 UTC. Логи вузлів збережено в `reports/logs/mongo-node{1,2,3}.log`.

2. `scripts/task4.sh` перезапускає replica set з логами в `reports/logs/task4/` і тричі проганяє `w1` і `majority`. Потім тричі прогони з kill: primary отримує SIGKILL на 35 % медіанного часу прогону без відмови (близько 8,1 с для `w1` і 35,7 с для `majority`). Після них один контрольний прогін без `lsid` і `txnNumber`. До і після кожного прогону скрипт читає `serverStatus().transactions.retriedStatementsCount` усіх вузлів: приріст показує, скільки повторів сервер упізнав як уже виконані. Для кожного прогону з kill він зберігає вивід `mongo-bench` і відповідні рядки журналів усіх трьох вузлів у `reports/logs/task4-failover-<прогін>-<повтор>.txt`. Результат записується в `reports/data/task4-bench.csv` (18:25…18:47 UTC).
3. Той самий скрипт запускає `counter-server --store mongo` і `loadgen` з 10 клієнтами по 10 000 запитів, тричі (18:47…18:56 UTC). Результат записується в `reports/data/task4-web.csv`. Числа для інших сховищ узято з `reports/data/task1.csv` (рядки з 10 клієнтами), `task2-web.csv` і `task3-web.csv`.
4. У таблицях наведено медіану трьох повторів, мінімум і максимум.

### 3.8 Аналітичний ноутбук

Дані оброблено в ноутбуці `solution.ipynb`, опублікованому разом зі звітом. Ноутбук показує протоколи частини I і витягує з них час і стани вузлів, читає `task4-bench.csv`, `task4-web.csv`, `task1.csv`, `task2-web.csv` і `task3-web.csv`, рахує в pandas медіани й діапазони для таблиць 4.1…4.4 і перевіряє, що вони збігаються зі звітом. Він розбирає журнали відмов (`reports/logs/task4-failover-*.txt`) і будує рисунки 4.1…4.3. Розділ «Жива перевірка» при `RUN_LIVE = True` піднімає replica set з трьох вузлів у тимчасовому каталозі, проганяє `mongo-bench` з `w: 1` і `w: "majority"` на 10 × 300 інкрементах і зупиняє вузли. Обидва прогони дали рівно 3 000. Числа звіту взято не з цього розділу, а з CSV.

## 4 Результати

### 4.1 Частина I: що показали сценарії

Write concern визначає лише момент, коли сервер відповідає клієнту, а не те, чи відбувся запис. У сценарії 1 вставка з `w: 3` чекала 10,623 с, з них близько 1,6 с після `start-node 3`: стільки вузол стартував і дотягнув документ з oplog. Якби вузол не повернувся, клієнт без `wtimeout` чекав би безкінечно. Весь цей час документ уже був на primary і на node2 і читався з `readConcern("majority")`: majority-committed означає «є на 2 з 3 вузлів», а не «підтверджено клієнту».

У сценарії 2 `WriteConcernTimeout` прийшов через 5,049 с, але документ залишився: є на primary, на node2 і після повернення на node3. За документацією MongoDB після `wtimeout` не скасовує вже виконаних змін [6], тому повтор такої вставки створив би другий документ. Документ був на 2 вузлах з 3, і тому читається з `readConcern: "majority"` [7].

У сценарії 3 штатна зупинка primary (SIGTERM) призвела до передачі ролі: `lastElectionReason: 'stepUpRequestSkipDryRun'`, вибори відбулися через 0,029 с після команди,

`w: "majority"` став доступний ще через 0,028 с. При раптовому SIGKILL (частина II) такої передачі немає, і secondary чекають `electionTimeoutMillis`, тобто 10 с. Старий primary повернувся як SECONDARY у term 2, наздогнав новий primary за 1,7 с і отримав три документи, записані без нього. Пріоритети вузлів однакові, тому роль primary лишилась на node2.

## 4.2 Частина II: час і точність лічильника

Таблиця 4.1. Час і пропускна здатність, 10 клієнтів по 10 000 інкрементів (медіана трьох повторів; контрольний прогін один)

| Прогін | Медіана, с | Мін., с | Макс., с | Інкр./с |
|---|---|---|---|---|
| `w1` | 22,49 | 22,00 | 29,18 | 4 447,0 |
| `majority` | 101,52 | 100,08 | 103,41 | 985,1 |
| `w1-kill` | 34,66 | 34,48 | 35,73 | 2 885,2 |
| `majority-kill` | 112,36 | 109,33 | 139,85 | 890,0 |
| `majority-kill-noretry` | 168,40 | 168,40 | 168,40 | 593,8 |

Таблиця 4.2. Фінальне значення, втрачені підтверджені інкременти, повтори клієнта і повтори, упізнані сервером, у кожному прогоні (очікується 100 000)

| Прогін | Фінальне значення | Втрачено | Повтори | Упізнано сервером |
|---|---|---|---|---|
| `w1` | 100 000 у всіх трьох | 0; 0; 0 | 0; 0; 0 | 0; 0; 0 |
| `majority` | 100 000 у всіх трьох | 0; 0; 0 | 0; 0; 0 | 0; 0; 0 |
| `w1-kill` | 99 996; 99 999; 100 000 | 4; 1; 0 | 480; 490; 450 | 0; 2; 0 |
| `majority-kill` | 100 000 у всіх трьох | 0; 0; 0 | 470; 480; 426 | 10; 7; 9 |
| `majority-kill-noretry` | 100 010 | 10 зайвих | 450 | 0 |

Без відмов обидва write concern дають рівно 100 000: `findOneAndUpdate` з `$inc` атомарно змінює документ на primary, тому втрачених оновлень, як у `IMap` без блокувань у роботі № 3, тут немає. `majority` у 4,5 раза повільніший за `w1` (101,52 проти 22,49 с). Середня затримка інкремента (10 клієнтів, поділені на пропускну здатність) 2,25 мс з `w: 1` і 10,15 мс з `w: "majority"`. З `w: 1` primary відповідає, щойно застосував запис у пам’яті. `w: "majority"` за замовчуванням включає `j: true`, тобто запис має потрапити в журнал на диску на більшості вузлів [6, 9], а `fsync` на цьому SSD займає 5,5…8,1 мс (робота № 1). Окремий `fsync` на кожен інкремент не потрібен: очікування write concern відбувається вже після того, як `$inc` застосовано, тож 10 клієнтів чекають спільних скидань журналу паралельно. 985,1 інкр./с у 8 разів більше, ніж дає файл з `fsync` на кожен інкремент (122,2 запитів/с).

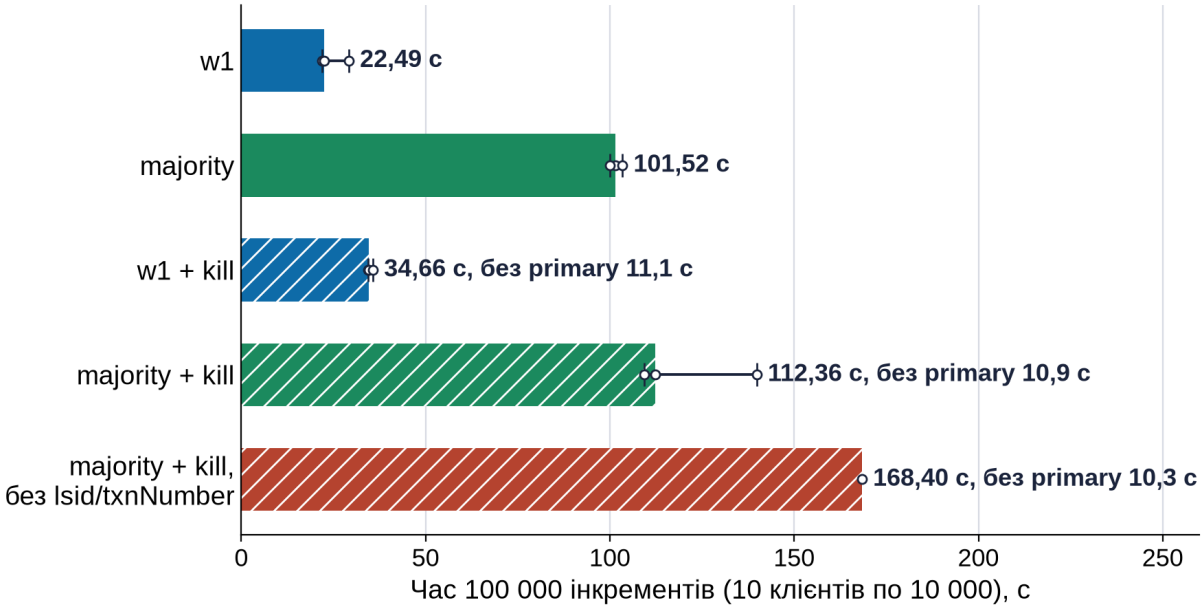



Рисунок 4.1. Час 100 000 інкрементів: медіана трьох повторів (стовпці), мінімум і максимум (відрізки), окремі повтори (точки); у прогонах з kill підписано медіану паузи без primary

Відмова primary коштує близько 11 с простою: від kill до першого підтвердженого інкременту 10,3…11,3 с у всіх семи прогонах. Secondary не отримують heartbeat від primary, чекають `electionTimeoutMillis` (10 000 мс) і проводять вибори. Прогони з kill довші за звичайні на 12,17 с (`w1`) і 10,84 с (`majority`) за медіаною. Усі 426…490 повторів припадають на цю паузу: 10 інкрементів, що були в дорозі, обриваються з `connectionClosed`, а далі кожен клієнт приблизно 4 рази за секунду отримує `noAvailableHosts`, доки primary немає. Помилок, на яких клієнт здався, немає в жодному прогоні. Великий розкид `majority-kill` (до 139,85 с) і повільний контрольний прогін (168,40 с) не пов’язані з виборами, пауза в них та сама (10,3…11,1 с). Імовірна причина в диску: розділ майже заповнений, а затримка `fsync` на ньому вже в роботі № 1 мала два рівні.

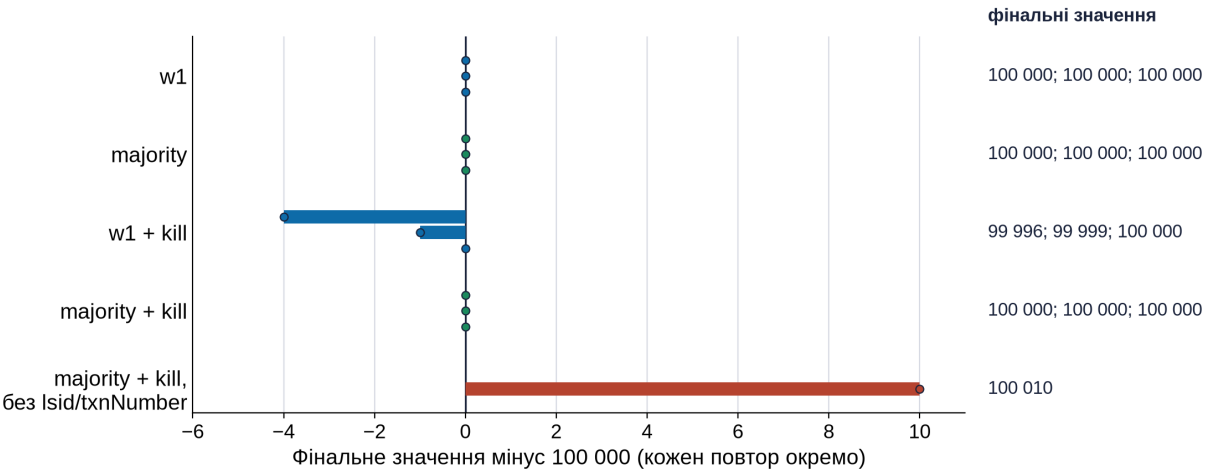



Рисунок 4.2. Відхилення фінального значення від 100 000 у кожному повторі: нуль означає точний лічильник, мінус втрачені підтверджені інкременти, плюс інкременти, пораховані двічі

## 4.3 Відмова primary: що стається з даними

Таблиця 4.3. Прогони з kill: вбитий вузол, новий primary і розрив між oplog нового primary (`targetOpTime` після виборів) і тим, що вбитий вузол мав на диску (`oplogTruncateAfterPoint` після перезапуску)

| Прогін | Повтор | Kill, с | Вбито | Новий primary | Розрив, записів oplog | Втрачено |
|---|---|---|---|---|---|---|
| `w1-kill` | 1 | 8,08 | node2 | node1, term 16 | 452 | 4 |
| `w1-kill` | 2 | 8,09 | node2 | node1, term 18 | 2 386 | 1 |
| `w1-kill` | 3 | 8,09 | node2 | node3, term 20 | 346 | 0 |
| `majority-kill` | 1 | 35,67 | node1 | node2, term 17 | 2 | 0 |
| `majority-kill` | 2 | 35,67 | node1 | node2, term 19 | 8 | 0 |
| `majority-kill` | 3 | 36,01 | node3 | node2, term 21 | 0 | 0 |

`w1-kill` втратив 4, 1 і 0 підтверджених інкрементів, разом 5 з 300 000. Як це стається, видно з таблиці 4.3 і журналів (уривок нижче). З `w: 1` primary відповідає клієнту до реплікації і до запису в журнал, а журнал без `j: true` скидається на диск раз на 100 мс [9]. Secondary натомість журналюють кожну застосовану пачку oplog. Тому після SIGKILL вбитий primary мав на диску на 346…2 386 записів oplog менше, ніж новий primary. Інкременти, які він встиг підтвердити, але не встиг віддати жодному secondary, не лишилися ніде: їх немає ні на новому primary, ні в журналі старого. Через це немає і rollback [12]: у журналах жодного рядка ROLLBACK, файлів rollback немає. Старий primary після перезапуску наздогнав новий як SECONDARY. Якби він встиг записати ці інкременти в журнал, вони стали б розбіжною гілкою oplog і були б відкочені в rollback-файли, але в лічильнику їх однаково не було б.

Уривок журналів трьох вузлів для `w1-kill`, повтор 1 (UTC, атрибути скорочено):

```
18:35:05.542 node1 Member is now in state DOWN  127.0.0.1:21431
18:35:16.556 node1 Starting an election, since we've seen no PRIMARY
             in election timeout period  {electionTimeoutPeriodMillis:10000}
18:35:16.561 node1 Election succeeded, assuming primary role  {term:16}
18:35:16.562 node1 Caught up to the latest optime known via heartbeats
             {targetOpTime: 1793644505:11766}
18:35:32.102 node2 ***** SERVER RESTARTED *****
18:35:32.856 node2 Incrementing the rollback ID after unclean shutdown
18:35:32.863 node2 Removing unapplied oplog entries after
             oplogTruncateAfterPoint  {1793644505:11314}
18:35:34.781 node2 Replica set state transition SECONDARY
```

`majority-kill` дав рівно 100 000 у всіх трьох повторах. Запис з `majority` підтверджується лише тоді, коли він є в журналі більшості вузлів, а нового primary можна

обрати лише з вузлів, що мають усі majority-committed записи. Тому жоден підтверджений інкремент не може зникнути. Розрив oplog тут 0…8 записів, бо кожен такий запис примусово синхронізує журнал [6].

Ще є 10 інкрементів, які були в дорозі в момент kill: клієнт не отримав відповіді і не знає, чи їх застосовано. Сервер упізнав 10, 7 і 9 з них як уже виконані: вони встигли потрапити на secondary, що став primary, разом з записом сесії в `config.transactions`, і новий primary повернув збережений результат. Решту (0, 3 і 1) новий primary виконав уперше. У контрольному прогоні без `lsid` і `txnNumber` сервер не зміг відрізнити ті самі 10 повторів від нових записів, і лічильник показав 100 010. Для точного лічильника при відмові primary потрібні і `w: "majority"`, і retryable writes.

## 4.4 Частина III: порівняння сховищ веб-лічильника

Таблиця 4.4. Веб-лічильник, 10 клієнтів по 10 000 запитів `GET /inc` (медіана трьох повторів, у всіх прогонах фінальне значення 100 000)

| Сховище | Загальний час, с | Мін., с | Макс., с | Запитів/с |
|---|---|---|---|---|
| In-memory | 3,20 | 2,92 | 3,31 | 31 256,0 |
| Disk (`fsync`) | 818,08 | 814,26 | 824,12 | 122,2 |
| PostgreSQL | 302,44 | 199,76 | 306,17 | 330,6 |
| Hazelcast (CP) | 8,88 | 8,79 | 9,08 | 11 259,7 |
| MongoDB (`majority`) | 133,59 | 121,80 | 156,01 | 748,6 |

Дані для пам’яті й диска взято з `reports/data/task1.csv`, для PostgreSQL з `task2-web.csv`, для Hazelcast з `task3-web.csv`, для MongoDB з `task4-web.csv`.

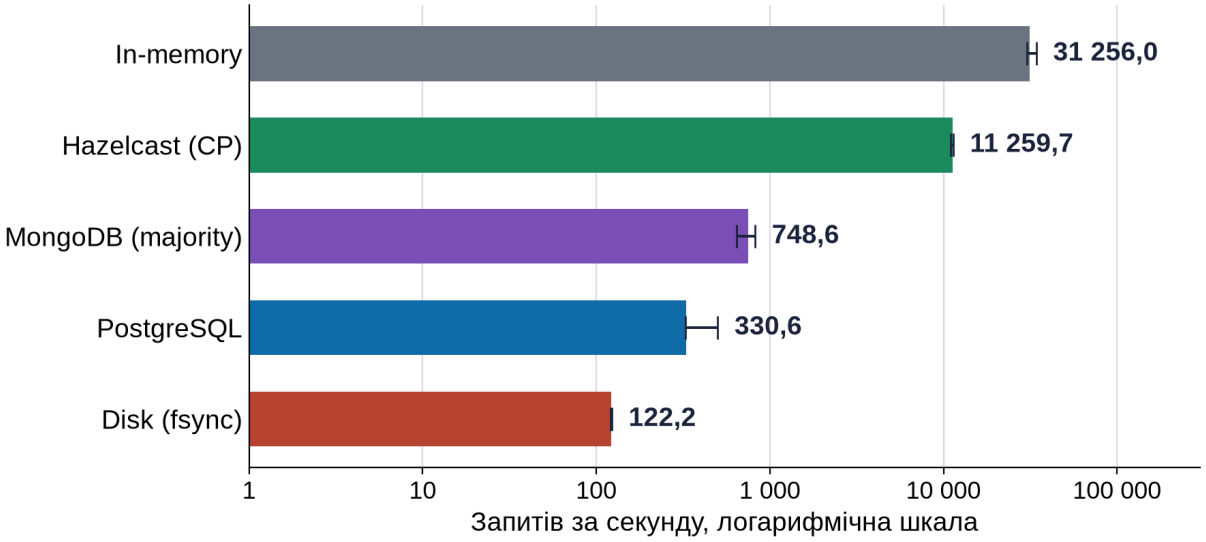



Рисунок 4.3. Пропускна здатність веб-лічильника для п’яти сховищ, логарифмічна шкала: медіана трьох повторів (стовпці), мінімум і максимум (відрізки)

MongoDB з `w: "majority"` дав 748,6 запитів/с. Це в 2,26 раза більше за PostgreSQL і в 6,1 раза більше за файл з `fsync`. Усі три сховища чекають на диск при кожному інкременті, різниця в тому, хто чекає паралельно. У файловому сховищі `Mutex` тримається разом з `fsync`. У PostgreSQL блокування рядка тримається до `COMMIT`, а `COMMIT` чекає `fsync` WAL, тож інкременти одного рядка йдуть строго по черзі (робота № 2). У MongoDB `$inc` застосовується і звільняє документ одразу, а очікування журналу на більшості вузлів іде вже після цього, тому 10 запитів чекають одночасно.

MongoDB у 15 разів повільніший за Hazelcast і в 41,8 раза за пам’ять. Hazelcast без persistence підтверджує запис після реплікації в пам’ять інших вузлів, а MongoDB з `majority` після запису в журнал на диску двох вузлів з трьох. Тому дані MongoDB зберігаються навіть після вимкнення всієї машини, а CP Subsystem Hazelcast без persistence витримує лише втрату вузла.

Веб-варіант повільніший за прямий `mongo-bench majority` (748,6 проти 985,1 інкр./с) через HTTP між `loadgen` і `counter-server`. Розкид між повторами від 121,80 до 156,01 с (25,6 % від медіани, у Hazelcast 3 %). Причина, як і в `majority-kill`, у затримці `fsync` на майже заповненому диску.

# 5 Додаткове дослідження

Розділ доповнює роботу трьома дослідженнями: справжній rollback і аномалії читання, пропускна здатність лічильника з `w: "majority"`, керована зміна primary і налаштування виборів. Виміри зроблено 10.11.2026 з 11:52 до 15:16 UTC на тій самій машині, але Nix flake на той час давав MongoDB 8.3.13 і mongosh 2.14.0, а розділ диска був заповнений на 86 %. Машину паралельно навантажували інші процеси (load average на початку прогонів від 2,55 до 22,7), тому варіанти, які порівнюються між собою, виконувались почергово (A, B, A, B, …) по три рази, і в таблицях наведено медіану. Числа розділів 3 і 4 не змінювались. Усі таблиці й рисунки цього розділу ноутбук перераховує з файлів `reports/data/task4x-*.csv` і протоколів `reports/logs/task4x-*.txt` і звіряє з текстом.

Таблиця 5.1. Що додано до програм і скриптів (без нових параметрів поведінка і рядок CSV ті самі, що в розділах 3 і 4)

| Програма | Додано |
|---|---|
| `scripts/mongo.sh` | `MONGO_TEST_COMMANDS=1` (вузли з `enableTestCommands`, потрібно для failpoint), `pause-node N` і `resume-node N` (SIGSTOP і SIGCONT), `MONGOD_ARGS` (додаткові параметри mongod), `MONGO_ELECTION_TIMEOUT_MS` (`rs.reconfig` з іншим `electionTimeoutMillis`) |
| `mongo-bench` | `--latency-out FILE` (рядок на кожен підтверджений інкремент), `--shards N` (лічильник з N документів), `--step-down-after T` (`replSetStepDown: 60` через T с) |
| `counter-server` | `--mongo-shards N` |
| нові скрипти | `task4x-rollback.sh`, `task4x-latency.sh` і `task4x-latency.py`, `task4x-throughput.py` і `task4x-journal-probe.py`, `task4x-failover.py` |

## 5.1 Відкат і аномалії читання

### 5.1.1 Rollback з rollback-файлами

У прогонах `w1-kill` rollback не було, бо вбитий primary не встиг записати підтверджені інкременти в журнал (таблиця 4.3). Скрипт `scripts/task4x-rollback.sh` будує випадок, коли підтверджені записи є в журналі старого primary, але на жодному secondary, і показує rollback [12]. Він піднімає окремий replica set з `MONGO_TEST_COMMANDS=1` і виконує кроки:

1. `insertMany` з `w: "majority"`: `{ _id: "counter", likes: 100 }` і документ `before` (рис. 5.1);
2. на обох secondary вмикається failpoint `stopReplProducer`, і вони перестають забирати oplog. Primary node1 отримує 5 разів `$inc likes` і 5 вставок `lost-1...lost-5`, кожен запис з `{ w: 1, j: true }` (рис. 5.2);
3. node1 заморожено (`pause-node 1`, SIGSTOP, як вузол за розривом мережі). Secondary обрали node3 у term 2, `wait-primary` знайшов його через 11,4 с. Після вимкнення failpoint node3 прийняв `$inc: { likes: 10 }` і документ `after` з `w: "majority"` (рис. 5.3);
4. node1 розморожено. Failpoint `rollbackHangBeforeStart` на ньому на 5 с затримує початок rollback, щоб прочитати дані в цьому стані (рис. 5.4), потім rollback відпущено (рис. 5.5 і 5.6).

Крок 6 протоколу (рис. 5.7) показує застаріле читання з відсталого secondary. Протокол записано в `reports/logs/task4x-rollback.txt`, повні журнали вузлів у `task4x-rollback-mongo-node{1,2,3}.txt`.

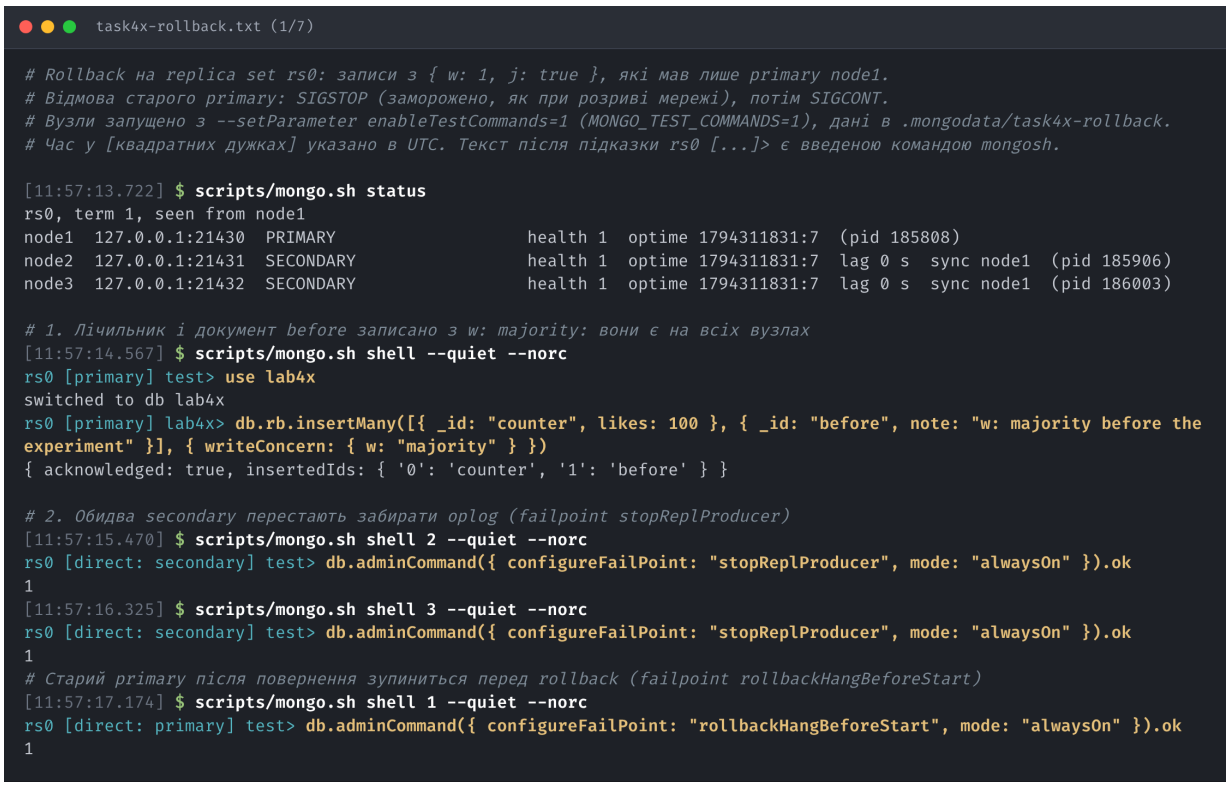



Рисунок 5.1. Rollback, частина 1: лічильник і документ `before` записано з `majority`, на secondary вимкнено отримання oplog

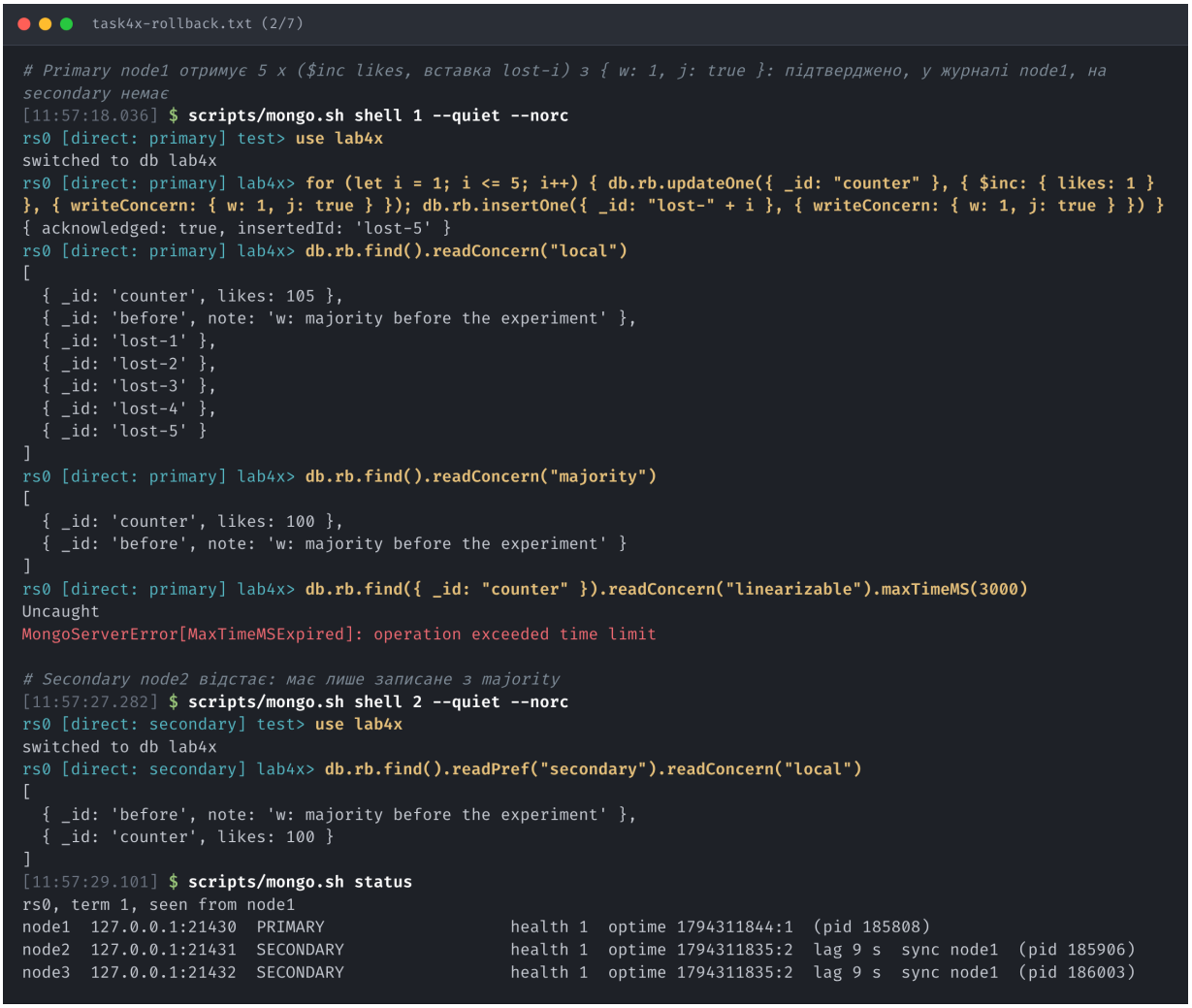



Рисунок 5.2. Rollback, частина 2: 10 записів з `{ w: 1, j: true }` на node1 і читання з `local`, `majority` і `linearizable`

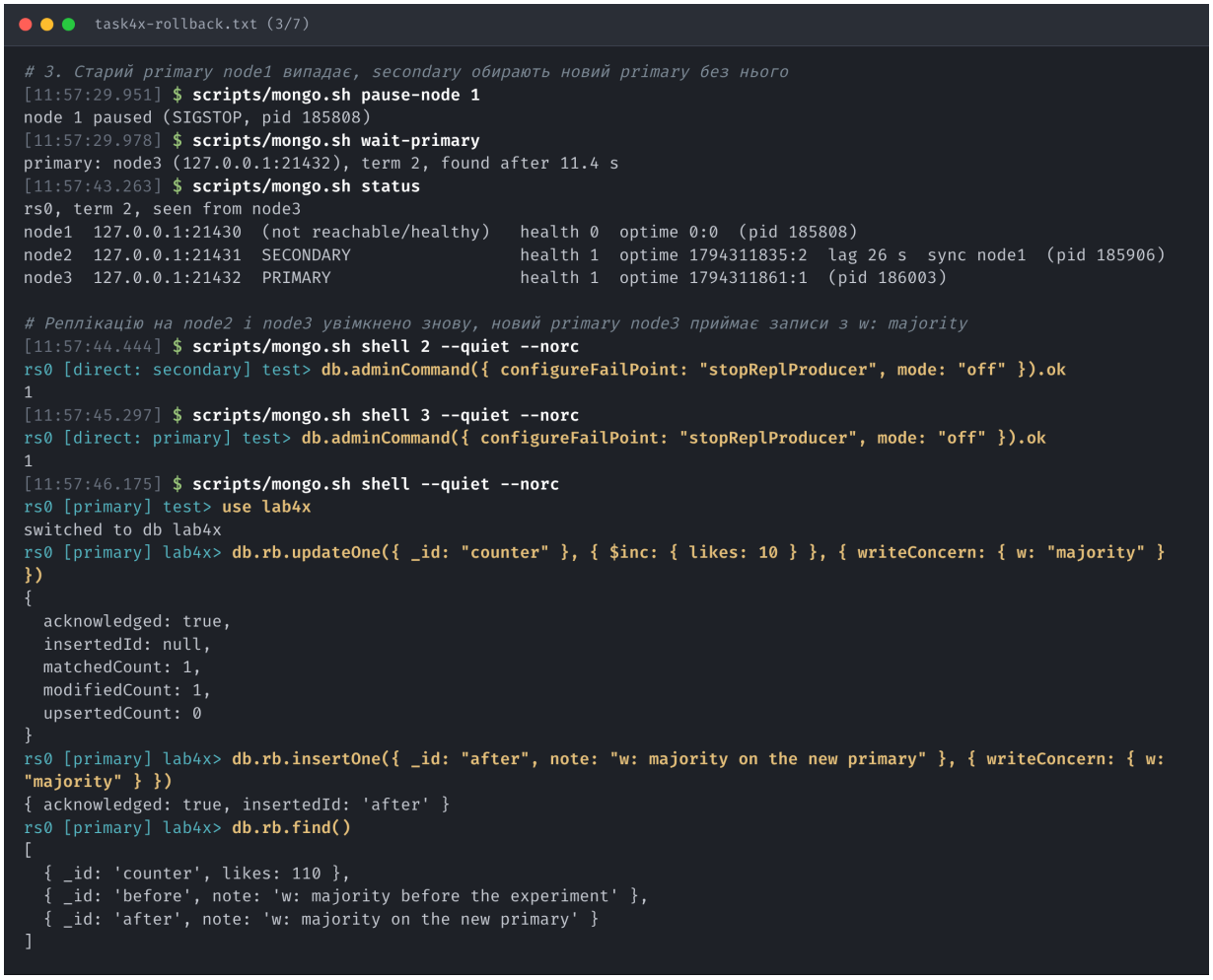



Рисунок 5.3. Rollback, частина 3: node1 заморожено, новий primary node3 приймає записи з `majority`

```
task4x-rollback.txt (4/7)

# 4. Старий primary node1 повертається
[11:57:50.658] $ scripts/mongo.sh resume-node 1
node 1 resumed (SIGCONT, pid 185808)
[11:57:50.692] $ scripts/mongo.sh status
rs0, term 2, seen from node3
node1  127.0.0.1:21430  SECONDARY                health 1  optime 1794311844:1  lag 23 s  sync node3  (pid 185808)
node2  127.0.0.1:21431  SECONDARY                health 1  optime 1794311867:1  lag 0 s  sync node3  (pid 185906)
node3  127.0.0.1:21432  PRIMARY                  health 1  optime 1794311867:1  (pid 186003)

# node1 уже SECONDARY, але rollback ще не почався: local повертає записи, які буде відкочено, majority їх не повертає
[11:57:51.577] $ scripts/mongo.sh shell 1 --quiet --norc
rs0 [direct: secondary] test> use lab4x
switched to db lab4x
rs0 [direct: secondary] lab4x> db.hello().secondary
true
rs0 [direct: secondary] lab4x> db.rb.find().readPref("secondary").readConcern("local")
[
  { _id: 'counter', likes: 105 },
  { _id: 'before', note: 'w: majority before the experiment' },
  { _id: 'lost-1' },
  { _id: 'lost-2' },
  { _id: 'lost-3' },
  { _id: 'lost-4' },
  { _id: 'lost-5' }
]
rs0 [direct: secondary] lab4x> db.rb.find().readPref("secondary").readConcern("majority")
[
  { _id: 'counter', likes: 100 },
  { _id: 'before', note: 'w: majority before the experiment' }
]

# Відпускаємо rollback
[11:57:54.000] $ scripts/mongo.sh shell 1 --quiet --norc
rs0 [direct: secondary] test> db.adminCommand({ configureFailPoint: "rollbackHangBeforeStart", mode: "off" }).ok
1
[11:57:56.110] $ scripts/mongo.sh wait-secondary 1
node1 is SECONDARY and caught up with node3 after 0.1 s (optime 1794311867:1)
```

Рисунок 5.4. Rollback, частина 4: node1 повернувся як SECONDARY, до початку rollback `local` ще показує записи, які буде відкочено

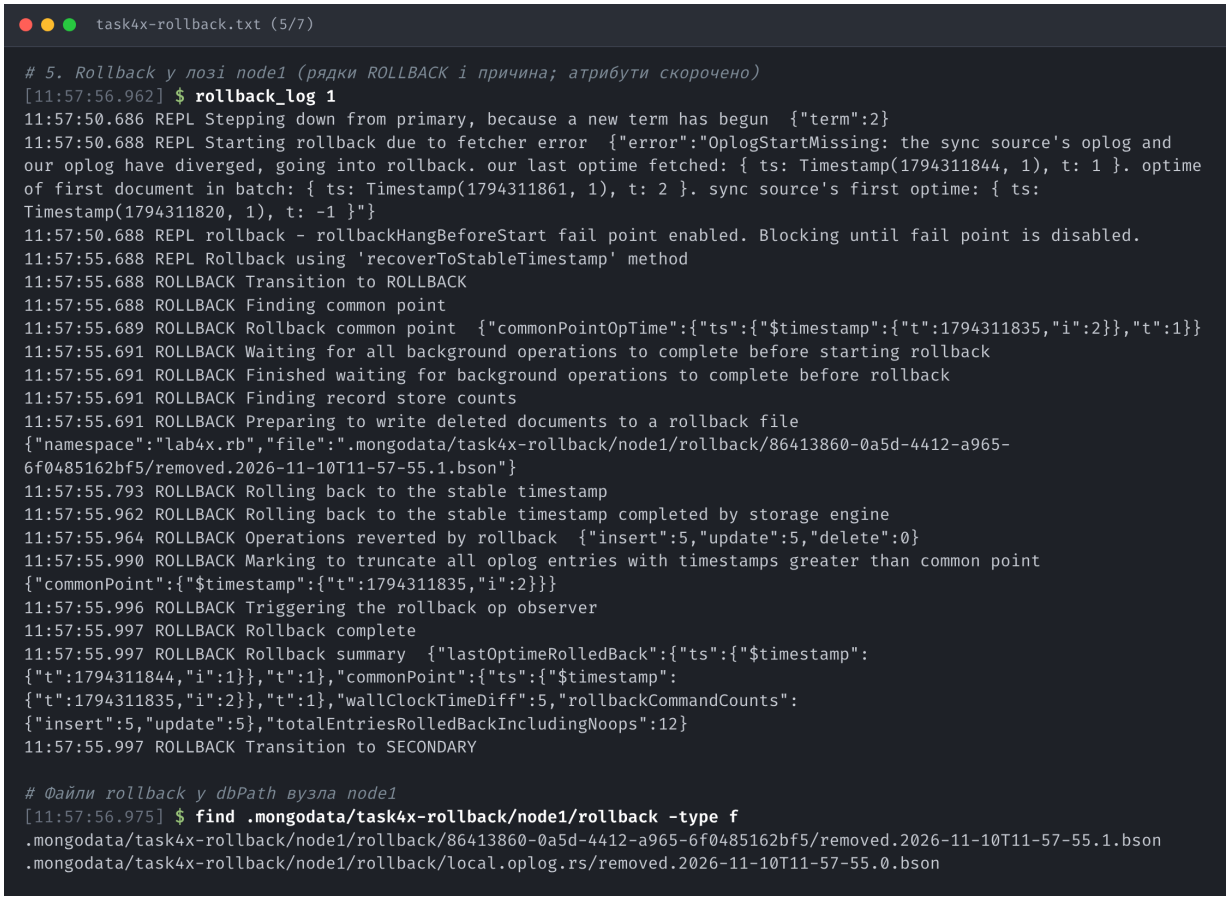



Рисунок 5.5. Rollback, частина 5: рядки ROLLBACK у журналі node1 і файли rollback

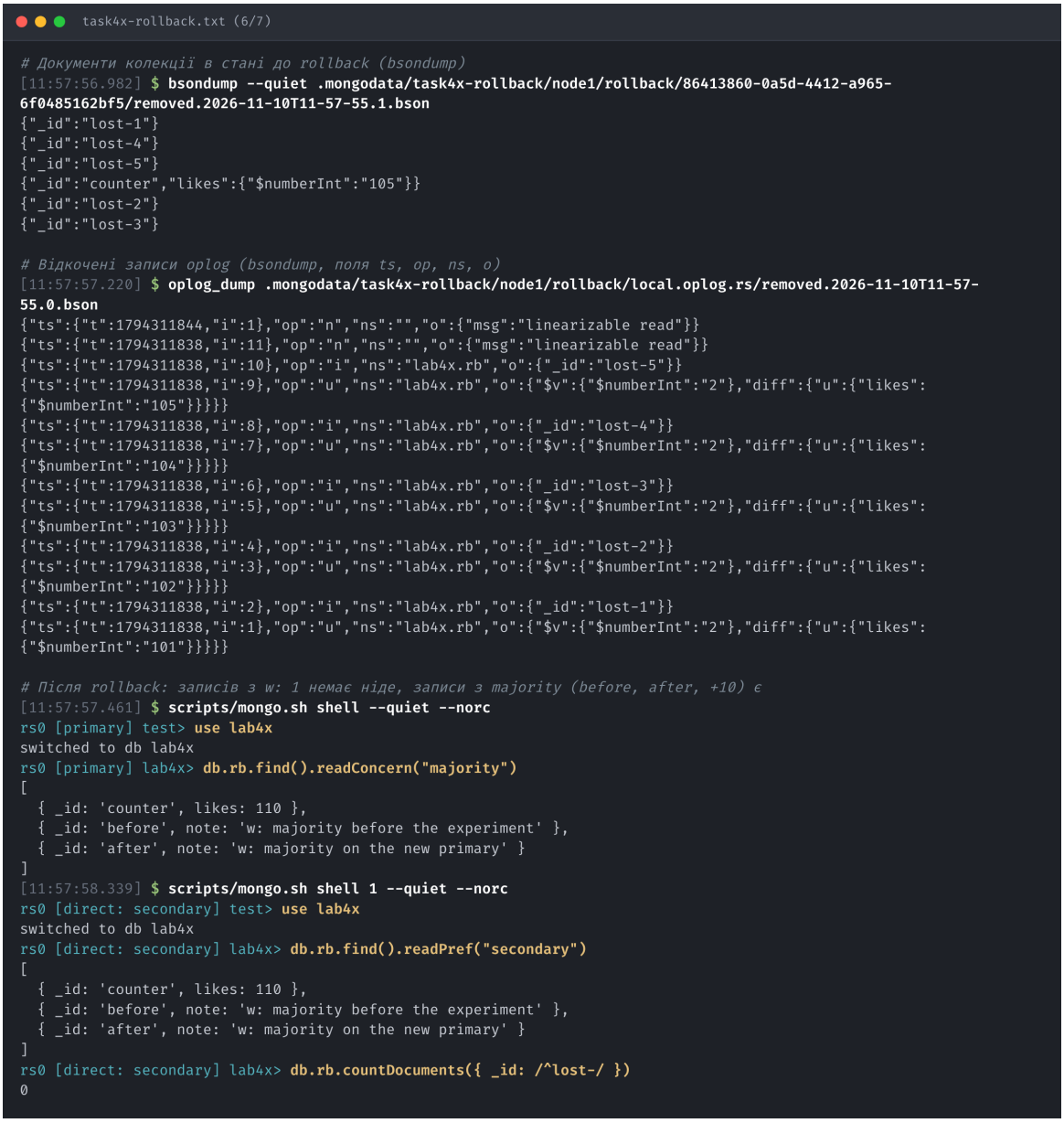



Рисунок 5.6. Rollback, частина 6: вміст rollback-файлів і дані після rollback

```
task4x-rollback.txt (7/7)

# 6. Застаріле читання: node2 перестає застосовувати oplog (rsSyncApplyStop), primary node3 отримує $inc з majority
[11:58:00.736] $ scripts/mongo.sh shell 2 --quiet --norc
rs0 [direct: secondary] test> db.adminCommand({ configureFailPoint: "rsSyncApplyStop", mode: "alwaysOn" }).ok
1
[11:58:01.583] $ scripts/mongo.sh shell --quiet --norc
rs0 [primary] test> use lab4x
switched to db lab4x
rs0 [primary] lab4x> db.rb.updateOne({ _id: "counter" }, { $inc: { likes: 1 } }, { writeConcern: { w: "majority" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
rs0 [primary] lab4x> db.rb.find({ _id: "counter" }).readConcern("linearizable")
[ { _id: 'counter', likes: 111 } ]
[11:58:04.128] $ scripts/mongo.sh shell 2 --quiet --norc
rs0 [direct: secondary] test> use lab4x
switched to db lab4x
rs0 [direct: secondary] lab4x> db.rb.find({ _id: "counter" }).readPref("secondary").readConcern("local")
[ { _id: 'counter', likes: 110 } ]
rs0 [direct: secondary] lab4x> db.rb.find({ _id: "counter" }).readPref("secondary").readConcern("majority")
[ { _id: 'counter', likes: 110 } ]
rs0 [direct: secondary] lab4x> db.rb.find({ _id: "counter" }).readPref("secondary").readConcern("linearizable")
Uncaught
MongoServerError[NotWritablePrimary]: cannot satisfy linearizable read concern on non-primary node
[11:58:07.276] $ scripts/mongo.sh status
rs0, term 2, seen from node3
node1  127.0.0.1:21430  SECONDARY                health 1  optime 1794311883:1  lag 0 s  sync node3  (pid 185808)
node2  127.0.0.1:21431  SECONDARY                health 1  optime 1794311881:1  lag 2 s  sync node3  (pid 185906)
node3  127.0.0.1:21432  PRIMARY                  health 1  optime 1794311883:1  (pid 186003)

# node2 знову застосовує oplog і наздоганяє primary
[11:58:08.143] $ scripts/mongo.sh shell 2 --quiet --norc
rs0 [direct: secondary] test> db.adminCommand({ configureFailPoint: "rsSyncApplyStop", mode: "off" }).ok
1
[11:58:08.985] $ scripts/mongo.sh wait-secondary 2
node2 is SECONDARY and caught up with node3 after 0.3 s (optime 1794311883:1)
[11:58:09.961] $ scripts/mongo.sh shell 2 --quiet --norc
rs0 [direct: secondary] test> use lab4x
switched to db lab4x
rs0 [direct: secondary] lab4x> db.rb.find({ _id: "counter" }).readPref("secondary").readConcern("local")
[ { _id: 'counter', likes: 111 } ]
```

Рисунок 5.7. Rollback, частина 7: застаріле читання з secondary, який не застосовує oplog

Журнал node1 (рис. 5.5) показує, як відбувся rollback. Через 2 мс після `Stepping down from primary, because a new term has begun` вузол пише `Starting rollback due to fetcher error` з `OplogStartMissing`: його oplog і oplog нового primary розійшлися. Після 5 с, які тримав failpoint, сам rollback від `Transition to ROLLBACK` до `Transition to SECONDARY` тривав 0,309 с і відкотив 5 вставок і 5 оновлень, разом 12 записів oplog. У каталозі `rollback/` вузла з'явилися два файли. Файл колекції після `bsondump` (рис. 5.6) містить 6 документів у стані до відкату: `lost-1...lost-5` і `{ _id: "counter", likes: 105 }`. Файл `local.oplog.rs` містить 12 відкочених записів oplog: 5 оновлень, 5 вставок і 2 no-op `linearizable read`, що їх залишили спроби linearizable читання. У MongoDB 8.3 каталог колекції названо її UUID, а не `<db>.<coll>`, і зв'язок з `lab4x.rb` видно лише з рядка `Preparing to write deleted documents to a rollback file`.

Після rollback усі вузли мають likes 110, документи `before` і `after`, а документів `lost-*` немає. Десять записів, підтверджених з `{ w: 1, j: true }`, зникли з колекції і лишились лише в rollback-файлах. `j: true` гарантує запис у журнал одного вузла, а від відкату захищає лише majority-committed запис.

Варіант з SIGKILL замість SIGSTOP (`FAILURE=kill`, протокол `task4x-rollback-kill.txt`) дав той самий результат (рис. 5.8): новий primary node2 через 10,6 с, rollback на 0,712 с з тими самими 5 вставками, 5 оновленнями і 12 записами oplog, ті самі 6 документів у rollback-файлі. Під час перезапуску node1 обрізав oplog після `oplogTruncateAfterPoint` 1794311948:1, і саме цей запис є `lastOptimeRolledBack` у підсумку rollback: усі записи з журналу відновились і лише потім були відкочені. Це підтверджує пояснення до таблиці 4.3: записи, що встигли в журнал, переживають SIGKILL і потрапляють у rollback-файли, а в прогонах `w1-kill` таких записів не було.

task4x-rollback-kill.txt (4/5)

```
# 5. Rollback у лозі node1 (рядки ROLLBACK і причина; атрибути скорочено)
[11:59:35.498] $ rollback_log 1
11:59:33.768 REPL Starting rollback due to fetcher error  {"error":"OplogStartMissing: the sync source's oplog and
our oplog have diverged, going into rollback. our last optime fetched: { ts: Timestamp(1794311948, 1), t: 1 }. optime
of first document in batch: { ts: Timestamp(1794311966, 2), t: 2 }. sync source's first optime: { ts:
Timestamp(1794311925, 1), t: -1 }"}
11:59:33.768 REPL Rollback using 'recoverToStableTimestamp' method
11:59:33.768 ROLLBACK Transition to ROLLBACK
11:59:33.768 ROLLBACK Finding common point
11:59:33.769 ROLLBACK Rollback common point  {"commonPointOpTime":{"ts":{"$timestamp":{"t":1794311940,"i":2}},"t":1}}
11:59:33.775 ROLLBACK Waiting for all background operations to complete before starting rollback
11:59:33.775 ROLLBACK Finished waiting for background operations to complete before rollback
11:59:33.775 ROLLBACK Finding record store counts
11:59:33.775 ROLLBACK Preparing to write deleted documents to a rollback file
{"namespace":"lab4x.rb","file":".mongodata/task4x-rollback/node1/rollback/5a2b9c1e-8a36-4096-b4f5-
e7ca06aa8da9/removed.2026-11-10T11-59-33.1.bson"}
11:59:33.877 ROLLBACK Rolling back to the stable timestamp
11:59:34.027 ROLLBACK Rolling back to the stable timestamp completed by storage engine
11:59:34.028 ROLLBACK Operations reverted by rollback  {"insert":5,"update":5,"delete":0}
11:59:34.076 ROLLBACK Marking to truncate all oplog entries with timestamps greater than common point
{"commonPoint":{"$timestamp":{"t":1794311940,"i":2}}}
11:59:34.480 ROLLBACK Triggering the rollback op observer
11:59:34.480 ROLLBACK Rollback complete
11:59:34.480 ROLLBACK Rollback summary  {"lastOptimeRolledBack":{"ts":{"$timestamp":
{"t":1794311948,"i":1}},"t":1},"commonPoint":{"ts":{"$timestamp":
{"t":1794311940,"i":2}},"t":1},"wallClockTimeDiff":5,"rollbackCommandCounts":
{"insert":5,"update":5},"totalEntriesRolledBackIncludingNoops":12}
11:59:34.480 ROLLBACK Transition to SECONDARY

# Файли rollback у dbPath вузла node1
[11:59:35.513] $ find .mongodata/task4x-rollback/node1/rollback -type f
.mongodata/task4x-rollback/node1/rollback/5a2b9c1e-8a36-4096-b4f5-e7ca06aa8da9/removed.2026-11-10T11-59-33.1.bson
.mongodata/task4x-rollback/node1/rollback/local.oplog.rs/removed.2026-11-10T11-59-33.0.bson
```

Рисунок 5.8. Rollback після SIGKILL старого primary: ті самі рядки ROLLBACK і файли rollback

### 5.1.2 Аномалії читання

Кожне читання з протоколу скрипт повторив окремим запитом і записав у файл `task4x-reads.csv` у теці `reports/data/` (таблиці 5.2 і 5.3) [7, 15].

Таблиця 5.2. Читання навколо відмови: значення `likes`, кількість документів `lost-*` і чи їх згодом відкочено

| Момент | Вузол | Read concern | likes | `lost-*` | Відкочено |
|---|---|---|---|---|---|
| записи з `w: 1` зроблено | node1, PRIMARY | `local` | 105 | 5 | так |
| те саме | node1, PRIMARY | `majority` | 100 | 0 | ні |
| те саме | node1, PRIMARY | `linearizable` | помилка | | |
| те саме | node2, SECONDARY | `local` | 100 | 0 | ні |
| новий primary обрано | node3, PRIMARY | `local` | 110 | 0 | ні |
| node1 повернувся, rollback затримано | node1, SECONDARY | `local` | 105 | 5 | так |
| те саме | node1, SECONDARY | `majority` | 100 | 0 | ні |
| після rollback | node1, SECONDARY | `local` | 110 | 0 | ні |
| після rollback | node3, PRIMARY | `majority` | 110 | 0 | ні |

Читання з `local` двічі повернуло дані, яких потім не стало: на старому primary до відмови і на ньому ж після повернення, коли `db.hello().secondary` уже `true`, але rollback ще не почався. Без failpoint друге вікно триває 2 мс, і клієнт з `readPreference: secondary` може в нього потрапити. `majority` в обох місцях повернув likes 100 без `lost-*`. `linearizable` на старому primary завершився помилкою: щоб переконатися, що вузол досі primary, сервер пише no-op і чекає його реплікації на більшість, а secondary не реплікували, тож запит завершився `MaxTimeMSExpired` через 3 с.

Таблиця 5.3. Застаріле читання: node2 забирає oplog, але не застосовує (failpoint `rsSyncApplyStop`), primary node3 отримав `$inc: { likes: 1 }` з `w: "majority"`

| Вузол | Read concern | likes |
|---|---|---|
| node3, PRIMARY | `linearizable` | 111 |
| node3, PRIMARY | `local` | 111 |
| node2, SECONDARY | `local` | 110 |
| node2, SECONDARY | `majority` | 110 |
| node2, SECONDARY | `linearizable` | `NotWritablePrimary` |
| node2 після вимкнення failpoint | `local` | 111 |

Відсталий secondary повертає старе значення 110 і з `local`, і з `majority`: `majority` гарантує, що прочитане не буде відкочено, але не що воно найсвіжіше. Останнє підтверджене значення гарантовано дає лише `linearizable`, і лише на primary.

### 5.1.3 Розподіл затримок і пауза при відмові

`mongo-bench --latency-out` для кожного підтвердженого інкременту записує момент завершення від початку прогону, затримку разом з повторами і кількість повторів (файли `task4x-latency-raw-*.csv.gz` у `reports/data/`). `scripts/task4x-latency.sh` виконав по одному прогону `w1`, `majority` і `majority-kill` на 10 × 10 000. Kill прийшов на 35 % часу прогону `majority` цього ж запуску, тобто на 34,1 с. Перцентилі пораховано за найближчим рангом.

Таблиця 5.4. Затримка інкремента, мс, по 100 000 значень на прогін (`task4x-latency.csv`)

| Прогін | Час, с | Інкр./с | Середня | p50 | p90 | p99 | p99,9 | max |
|---|---|---|---|---|---|---|---|---|
| `w1` | 12,74 | 7 852,1 | 1,241 | 0,413 | 1,701 | 20,104 | 59,299 | 141,5 |
| `majority` | 97,49 | 1 025,8 | 9,728 | 9,679 | 13,618 | 16,944 | 25,789 | 49,447 |
| `majority-kill` | 107,67 | 928,8 | 10,754 | 9,826 | 13,226 | 17,106 | 26,486 | 10 411,9 |

Усі три прогони дали рівно 100 000. `majority` близький до таблиці 4.1 (97,49 проти 101,52 с), а `w1` пройшов за 12,74 с замість 22,49 с. Причину цього я не досліджував. Від розділу 4 відрізняються версія mongod і стан диска.

З `w: 1` медіана 0,41 мс, але p99 у 49 разів більший, і 1 839 інкрементів (1,8 %) довші за 10 мс. З `majority` розподіл вузький, від 9,7 мс (p50) до 16,9 мс (p99), бо кожен інкремент чекає журналу. У `majority-kill` перцентилі до p99,9 такі самі, як без відмови. Довших за 1 с інкрементів лише 10, по одному на клієнта: це ті, що були в дорозі під час kill. Усі вони мали повтори і тривали від 10,397 до 10,412 с. Відмову primary видно в максимумі, а не в p99,9 (рис. 5.9).

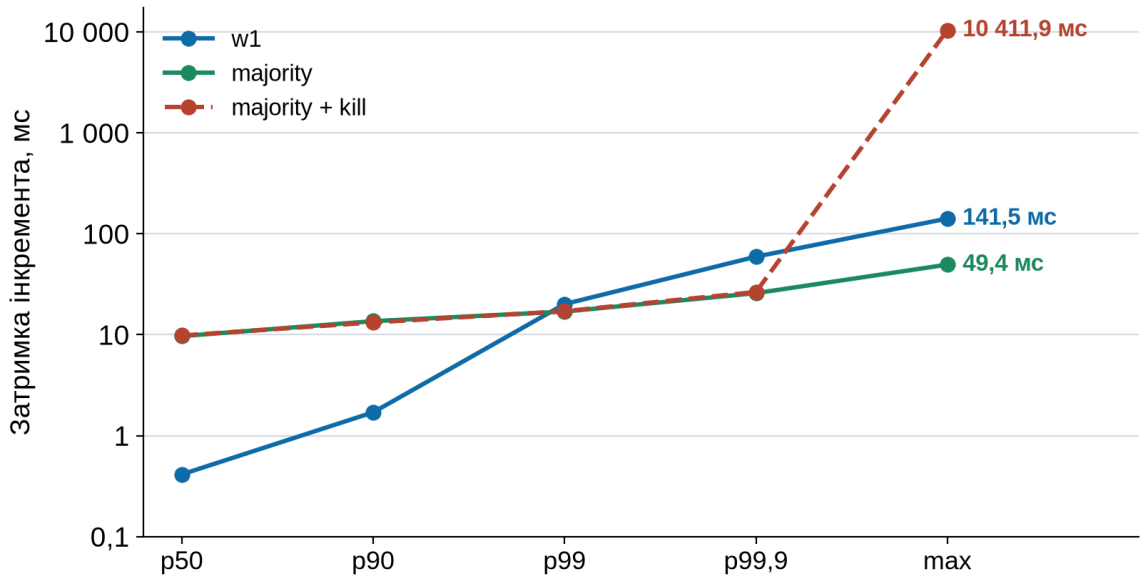



Рисунок 5.9. Перцентилі затримки інкремента для трьох прогонів, логарифмічна шкала

На часовій шкалі `majority-kill` (рис. 5.10) останній інкремент перед паузою підтверджено на 34,130 с, наступний через 10,397 с, і 103 інтервали по 100 мс від 34,2 до 44,4 с порожні. `mongo-bench` виміряв від kill (34,242 с) до першого підтвердження 10,285 с. За журналами вузли побачили primary недоступним о 11:54:53.175 UTC, а новий primary дозволив запис о 11:55:03.381, через 10,21 с. Клієнти зробили 450 повторів: 440 після `noAvailableHosts` і 10 після `connectionClosed`. Rollback у цьому прогоні не було.

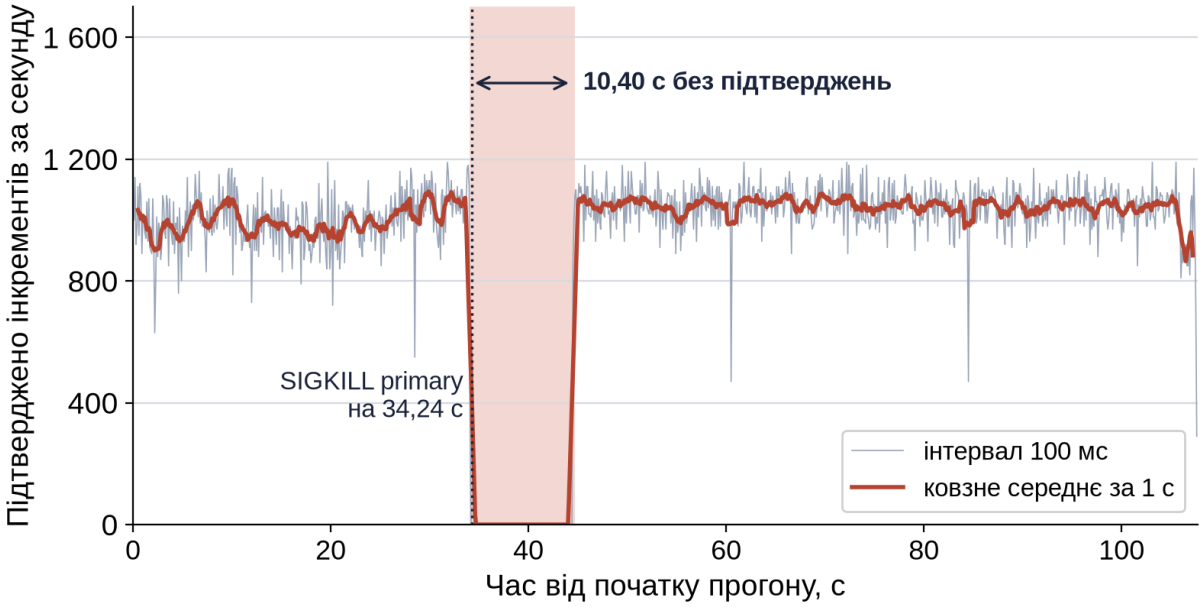



Рисунок 5.10. Підтверджені інкременти за секунду в прогоні `majority-kill`: інтервали по 100 мс і ковзне середнє за 1 с; затінено паузу без підтверджень

## 5.2 Пропускна здатність

Мета дослідження: підняти пропускну здатність лічильника з `w: "majority"` (985,1 інкр./с у таблиці 4.1 і 748,6 запитів/с у таблиці 4.4) без зміни правил. Кожен інкремент і далі є одним `findAndModify` з `$inc` і `w: "majority"`, `writeConcernMajorityJournalDefault` лишається `true`, тобто запис підтверджується лише після журналу на більшості вузлів. Клієнт той самий: 10 клієнтів, по одному запиту в польоті, без батчингу.

`scripts/task4x-throughput.py` для кожного варіанта піднімає replica set з потрібними параметрами mongod (`MONGOD_ARGS`) і робить розігрівний прогін. До і після кожного прогону він знімає `serverStatus` і час CPU всіх вузлів (`task4x-throughput-metrics.csv`): кількість і час синхронізацій журналу WiredTiger, конфлікти запису, `metrics.getLastError.wtime` (скільки primary чекав write concern), показники flow control і розмір пачок oplog на secondary. Розвідувальні серії йшли на 10 × 2 000, підсумкова на 10 × 10 000. Усі 93 прогони (`task4x-throughput.csv`) закінчились точним значенням.

Таблиця 5.5. До і після, 10 × 10 000, медіана трьох почергових прогонів, інкр./с (у веб-варіанті запитів/с)

| Режим | Звіт | До | Після | Мін…макс після | Зміна |
|---|---|---|---|---|---|
| `mongo-bench`, `majority` | 985,1 | 997,6 | 1 520,0 | 1 419,5…1 535,7 | +52,4 % |
| веб-лічильник, `majority` | 748,6 | 870,7 | 1 474,6 | 1 300,7…1 525,4 | +69,4 % |
| `mongo-bench`, `w: 1` | 4 447,0 | 4 641,1 | 5 228,1 | 4 658,4…5 959,5 | +12,6 % |

«Звіт» узято з таблиць 4.1 і 4.4, «до» є тією самою конфігурацією в цій серії, «після» відрізняється одним параметром: журнал WiredTiger заповнюється нулями під час створення файлу (`--wiredTigerEngineConfigString "log=(zero_fill=true)"` [16]). Веб-варіант використовує той самий `counter-server --engine async` і `loadgen`, що й таблиця 4.4. Для `w: 1` діапазон «до» (4 448,7…5 466,0) перетинається з діапазоном «після», тож приросту там немає.

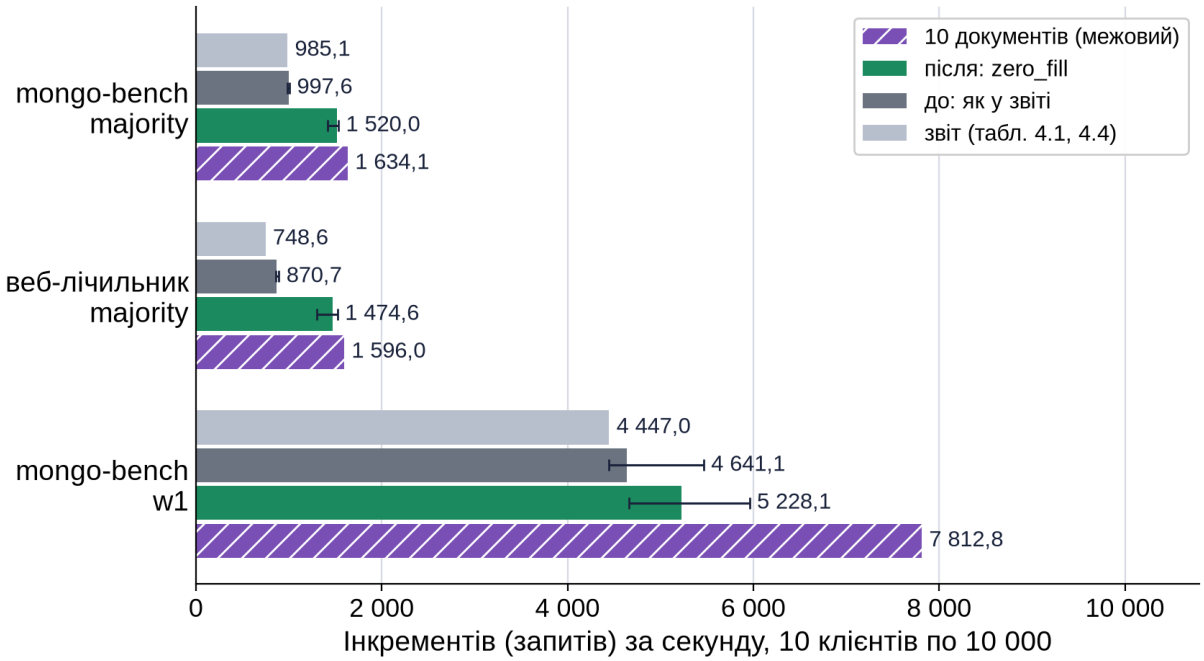



Рисунок 5.11. Пропускна здатність до і після `zero_fill`: медіана (стовпці), мінімум і максимум трьох прогонів (відрізки); лічильник з 10 документів заштриховано

### 5.2.1 Вузьке місце: синхронізація журналу

Таблиця 5.6. Метрики вузлів у підсумковій серії `majority`, медіана трьох прогонів (`task4x-throughput-metrics.csv`)

| Показник | До | Після (`zero_fill`) |
|---|---|---|
| інкр./с | 997,6 | 1 520,0 |
| затримка інкремента (10 клієнтів / інкр./с), мс | 10,02 | 6,58 |
| очікування write concern на primary (`wtime`), мс | 8,235 | 5,172 |
| синхронізацій журналу на інкремент, primary | 0,170 | 0,184 |
| час однієї синхронізації, primary, мс | 5,763 | 3,410 |
| синхронізацій журналу на інкремент, secondary | 0,212 | 0,208 |
| час однієї синхронізації, secondary, мс | 4,637 | 3,066 |
| зайнятість журналу primary, с за секунду | 0,98 | 0,95 |
| зайнятість журналу secondary, с за секунду | 0,98 | 0,97 |
| CPU primary, % одного ядра | 100 | 147 |

З 10,02 мс затримки інкремента 8,235 мс (82 %) primary чекає write concern, тобто доки запис буде в журналі на диску primary і одного secondary. Журнал кожного вузла при цьому синхронізується майже безперервно: на primary 0,170 × 997,6 = 170 синхронізацій за секунду по 5,763 мс, тобто 0,98 с синхронізації на кожну секунду, на secondary теж 0,98 с. Group commit уже працює: одна синхронізація покриває 5,9 інкремента на primary і 4,7 на secondary.

Ціну журналу показує небезпечна конфігурація, виміряна лише для оцінки: з `writeConcernMajorityJournalDefault: false` `majority` дав 3 367,1 інкр./с проти 884,3 у тій самій серії, а `wtime` упав з 9,35 до 0,64 мс. Сама реплікація (secondary забирає oplog, застосовує і повідомляє primary) коштує близько 0,6 мс, решта очікування припадає на синхронізацію журналу.

WiredTiger заздалегідь створює файли журналу по 100 МБ через `posix_fallocate`, і `filefrag -v` показує, що вони складаються з extents з позначкою `unwritten` (`reports/logs/task4x-throughput-filefrag.txt`). Перший запис у такий блок перетворює extent на записаний, тобто змінює метадані ext4 [17], тому `fdatasync` мусить закомітити ще й журнал файлової системи. Проба без MongoDB (`scripts/task4x-journal-probe.py`, таблиця 5.7) це підтверджує.

Таблиця 5.7. Дописування по 2 048 байтів з `fdatasync` після кожного, середній час однієї операції, мс (3 почергові раунди по 500, `task4x-journal-probe.csv`)

| Підготовка файла | 1 процес | 3 процеси одночасно |
|---|---|---|
| `posix_fallocate` (як у WiredTiger) | 3,435 | 4,051 |
| `ftruncate` (розріджений файл) | 3,446 | 3,989 |
| заповнений нулями | 1,468 | 2,650 |

У файлі, заповненому нулями, немає unwritten extents, і `fdatasync` записує лише дані. Параметр `log=(zero_fill=true)` робить так з кожним новим файлом журналу, і після запуску з ним `filefrag -v` не показує жодного unwritten extent. Час однієї синхронізації впав до 3,410 мс на primary і 3,066 мс на secondary, `wtime` до 5,172 мс (таблиця 5.6). Ціна параметра: кожен новий файл журналу на 100 МБ спершу записується нулями у фоновому потоці WiredTiger. Відновлення після збою з цим параметром окремо не перевірялось.

Журнал і після зміни зайнятий 95…97 % часу. Три вузли пишуть журнали на один NVMe, і проба з трьома процесами на заповненому нулями файлі дає 2,65 мс на синхронізацію. Тепер межу задає кількість скидань кешу за секунду, яку один SSD встигає для трьох вузлів.

Один клієнт mongosh без конкуренції (1 000 послідовних інкрементів, медіана трьох повторів, `task4x-throughput-probe.csv`) витрачає 1,150 мс з `w: 1`, 3,917 мс з `{ w: 1, j: true }` і 4,232 мс з `majority`, а з `zero_fill` 1,112, 3,442 і 3,660 мс. `majority` довший за `j: true` на одному primary лише на 0,2…0,3 мс, бо журнали primary і secondary синхронізуються паралельно.

### 5.2.2 Гарячий документ і лічильник з кількох документів

Таблиця 5.8. Один документ проти 10: серія `hotdoc` (10 × 2 000, медіана трьох прогонів) і профіль CPU primary з окремого прогону (`perf record -e cpu-clock:u`, 6 с, `task4x-throughput-perf.csv`)

| Навантаження | Інкр./с | Конфліктів запису на інкремент | CPU на інкремент, мкс | Розгортання винятків, % |
|---|---|---|---|---|
| `w: 1`, 1 документ | 4 676,6 | 3,89 | 538 | 14,1 |
| `w: 1`, 10 документів | 6 873,3 | 0,00 | 301 | 0,0 |
| `majority`, 1 документ | 882,3 | 3,57 | 881 | 7,6 |
| `majority`, 10 документів | 1 043,4 | 0,00 | 586 | 0,0 |

Коли всі 10 клієнтів змінюють один документ, на primary виникає 3,57…3,89 конфлікту запису WiredTiger на кожен інкремент. mongod повторює операцію сам після `WriteConflictException`, і обробку цього винятку видно в профілі як розгортання винятків C++ (`libgcc_s`). Для `w: 1` гарячий документ є головним обмеженням: 10 документів дають +47 %.

Для `majority` це +18 % у серії `hotdoc` і +7,5 % з `zero_fill` (таблиця 5.9), бо там основний час іде на журнал, а primary використовує близько одного ядра.

Таблиця 5.9. Межовий варіант: лічильник з 10 документів, 10 × 10 000 з `zero_fill`, медіана трьох прогонів, інкр./с

| Режим | 1 документ | 10 документів | Зміна |
|---|---|---|---|
| `mongo-bench`, `majority` | 1 520,0 | 1 634,1 | +7,5 % |
| веб-лічильник, `majority` | 1 474,6 | 1 596,0 | +8,2 % |
| `mongo-bench`, `w: 1` | 5 228,1 | 7 812,8 | +49,4 % |

Варіант межовий, бо змінює модель даних, як `postgres-sharded` у роботі № 2. Кожен інкремент і далі є одним `findAndModify` з `$inc` і `w: "majority"`, але клієнт і збільшує документ і mod 10, а значення є сумою 10 документів, які `value()` читає одним `find` з `$in` і `readConcern: "majority"`. Фінальне значення точне в усіх прогонах. До підсумку в таблиці 5.5 цей варіант не входить.

### 5.2.3 Що не дало приросту

Таблиця 5.10. Зміни без приросту: 10 × 2 000, медіана трьох почергових прогонів, контроль з тієї самої серії, інкр./с

| Зміна | Інкр./с | Контроль | Різниця |
|---|---|---|---|
| `journalCommitInterval` 1 мс (за замовчуванням 100) | 881,1 | 886,8 | -0,6 % |
| `journalCommitInterval` 500 мс | 882,7 | 886,8 | -0,5 % |
| `enableFlowControl: false` | 1 000,3 | 992,4 | +0,8 % |
| `replWriterThreadCount` 1 (замість 16) | 1 003,9 | 992,4 | +1,2 % |
| веб: `--mongo-clients 20` | 860,2 | 867,6 | -0,9 % |
| веб: `loadgen --engine nio` | 892,4 | 867,6 | +2,9 % |
| веб: `--engine handler`, `loadgen --engine nio` | 886,1 | 867,6 | +2,1 % |
| веб: `--engine raw`, `loadgen --engine nio` | 895,4 | 867,6 | +3,2 % |

Інтервал журналу задає лише періодичне скидання, а запис з `majority` викликає скидання одразу, коли його чекає: за будь-якого інтервалу на primary 0,156…0,157 синхронізації на інкремент. Flow control не вмикався в жодному з 93 прогонів (`timeAcquiringMicros` і `isLaggedCount` дорівнюють 0): він обмежує запис на primary лише тоді, коли majority-committed точка відстає більше ніж на `flowControlTargetLagSeconds` (10 с) [1]. Пачки oplog на secondary малі, 2,7…2,9 записи для одного документа, тому кількість потоків застосування не важлива. Кожен клієнт пулу має одне з’єднання з primary і один запит у польоті, і при 10 HTTP-з’єднаннях 10 клієнтів пулу досить. Інші рушії `counter-server` і `loadgen` дають до 3,2 %, у межах розкиду.

Розрив між веб-лічильником і mongo-bench у почерговій серії 12,0 % (867,6 проти 986,2), а не 24 %, як між таблицями 4.4 і 4.1, виміряними в різний час. Затримка через HTTP довша на 1,39 мс. З них 0,81 мс видно в mongod (wtime 9,03 проти 8,22 мс, синхронізація журналу primary 6,40 проти 5,67 мс), решта, близько 0,6 мс, припадає на HTTP. Чому інкременти через HTTP довше чекають журналу, я не встановив. З zero_fill розрив зменшився до 3,0 % (1 474,6 проти 1 520,0).

## 5.3 Керована зміна primary і налаштування виборів

mongo-bench --step-down-after T через T с знаходить primary командою hello і надсилає йому replSetStepDown: 60 [14]. Команда йде без повторів (новий метод MongoCounterClient.adminCommandOnce), бо повтор після обірваної відповіді міг би змусити скласти роль і новий primary. scripts/task4x-failover.py виконує прогони і з журналів mongod бере час від відмови до DOWN, початку виборів, обрання і дозволу на запис (reports/data/task4x-failover.csv, 22 прогони), а момент першого підтвердження бере з --latency-out. Новий інтеграційний тест перевіряє, що інкременти з w: 1 і w: "majority" лишаються точними через step-down primary. Це перший тест гілки reconnect() у клієнті, помилок у ній не знайдено.

Таблиця 5.11. replSetStepDown під навантаженням: 10 × 10 000, electionTimeoutMillis 10 000, step-down на 5-й с (w1) і на 30-й с (majority); час від надсилання команди до початку виборів, дозволу на запис, першого підтвердження і найдовший окремий інкремент

| Прогін | Повтор | Час, с | Повтори | Вибори, с | Запис, с | Підтв., с | Найдовший, с |
|---|---|---|---|---|---|---|---|
| w1 | 1 | 14,27 | 20 | 0,013 | 0,030 | 0,043 | 0,197 |
| w1 | 2 | 15,52 | 20 | 0,011 | 0,024 | 0,040 | 0,188 |
| w1 | 3 | 17,71 | 12 | 0,012 | 0,023 | 0,026 | 0,135 |
| majority | 1 | 99,06 | 17 | 0,010 | 0,018 | 0,023 | 0,044 |
| majority | 2 | 98,68 | 20 | 0,010 | 0,018 | 0,038 | 0,055 |
| majority | 3 | 98,77 | 11 | 0,009 | 0,018 | 0,025 | 0,053 |

Усі шість прогонів дали рівно 100 000 при 5 646,7…7 006,5 інкр./с з w: 1 і 1 009,5…1 013,4 інкр./с з majority. Пауза в записі після step-down становить 0,023…0,043 с проти 10,1…10,9 с після SIGKILL з тим самим electionTimeoutMillis (таблиця 5.13). replSetStepDown відповідає за 0,010…0,014 с: старий primary переходить у SECONDARY і передає вибори наздоганому secondary (Handing off election, replSetStepUp), а той починає вибори без пробного туру. Кожен з 10 клієнтів отримує одну помилку InterruptedDueToReplStateChange (код 11602) для інкременту, що був у дорозі, і повторює його з тим самим txnNumber. У majority це здебільшого writeConcernError: $inc уже застосовано, перервано лише очікування. Решта повторів (від 1 до 10 на прогін) це noAvailableHosts, поки новий primary ще не готовий. На відміну від w1-kill, w1 тут точний, бо replSetStepDown чекає,

доки secondary наздожене primary, і роль отримує саме такий вузол [14].

Таблиця 5.12. Контроль: той самий step-down у збірці, де закоментовано рядок `try? await cluster.reconnect()` (після прогонів його повернуто)

| Прогін | Повтор | Час, с | Перше підтв., с | Найдовший, с | Повтори (з них `NotWritablePrimary`) |
|---|---|---|---|---|---|
| `w1` | 1 | 18,86 | 5,072 | 5,199 | 240 (230) |
| `w1` | 2 | 20,59 | 5,072 | 5,129 | 240 (230) |
| `w1` | 3 | 20,44 | 5,075 | 5,127 | 240 (230) |
| `majority` | 1 | 104,53 | 0,033 | 10,088 | 146 (132) |
| `majority` | 2 | 104,42 | 0,032 | 10,083 | 104 (87) |
| `majority` | 3 | 99,87 | 0,032 | 0,047 | 20 (1) |

Без `reconnect()` старий primary відповідає на кожен повтор `NotWritablePrimary`, а MongoKitten вважає його primary, доки сам не перевірить вузли. Він робить це раз на 10 с від створення клієнта. Клієнти створено на початку прогону, а step-down був на 5-й секунді, тому всі клієнти `w1` стояли 5,07 с. У `majority` step-down на 30-й секунді майже збігся з черговою перевіркою: частина клієнтів продовжила одразу, інші стояли до наступної перевірки, 10,08 с. Фінальне значення точне і без `reconnect()`, змінюється лише пауза: з ним найдовший інкремент становить 0,044…0,197 с.

Таблиця 5.13. SIGKILL primary на 30-й с прогону `majority` (10 × 10 000), по 3 прогони на кожен таймаут виборів (`electionTimeoutMillis`); час від SIGKILL, мінімум…максимум

| Таймаут, мс | Вибори почались, с | Запис дозволено, с | Перше підтвердження, с (медіана) | Прогін, с (медіана) | Повтори |
|---|---|---|---|---|---|
| 10 000 | 10,053…10,805 | 10,062…10,820 | 10,144…10,910 (10,144) | 106,93 | 440…470 |
| 2 000 | 2,015…2,096 | 2,030…2,107 | 2,096…2,353 (2,112) | 98,76 | 120…130 |
| 1 000 | 1,006…1,131 | 1,021…1,146 | 1,101…1,339 (1,105) | 98,12 | 80…90 |

`electionTimeoutMillis` є полем `settings` конфігурації replica set [13], і `MONGO_ELECTION_TIMEOUT_MS` змінює його через `rs.reconfig`. Майже весь простій займає очікування цього таймауту. Вибори починаються через `electionTimeoutMillis` плюс випадкова добавка [4]: 53…805 мс при 10 000, 15…96 мс при 2 000, 6…131 мс при 1 000. Вибори і перехід до запису займають 9…19 мс, а від дозволу на запис до першого підтвердження минає 0,001…0,246 с: це пауза клієнта між повторами, що зростає від 10 до 250 мс. Вузли позначають вбитий primary як DOWN уже через 0,032…0,045 с, бо наступна спроба heartbeat дістає `Connection refused`. Вибори це не прискорює: secondary чекають `electionTimeoutMillis` від

останнього повідомлення primary. У кожному прогоні були рівно одні вибори і рівно 100 000, зайвих виборів під навантаженням не було і при 1 000 мс (рис. 5.12).

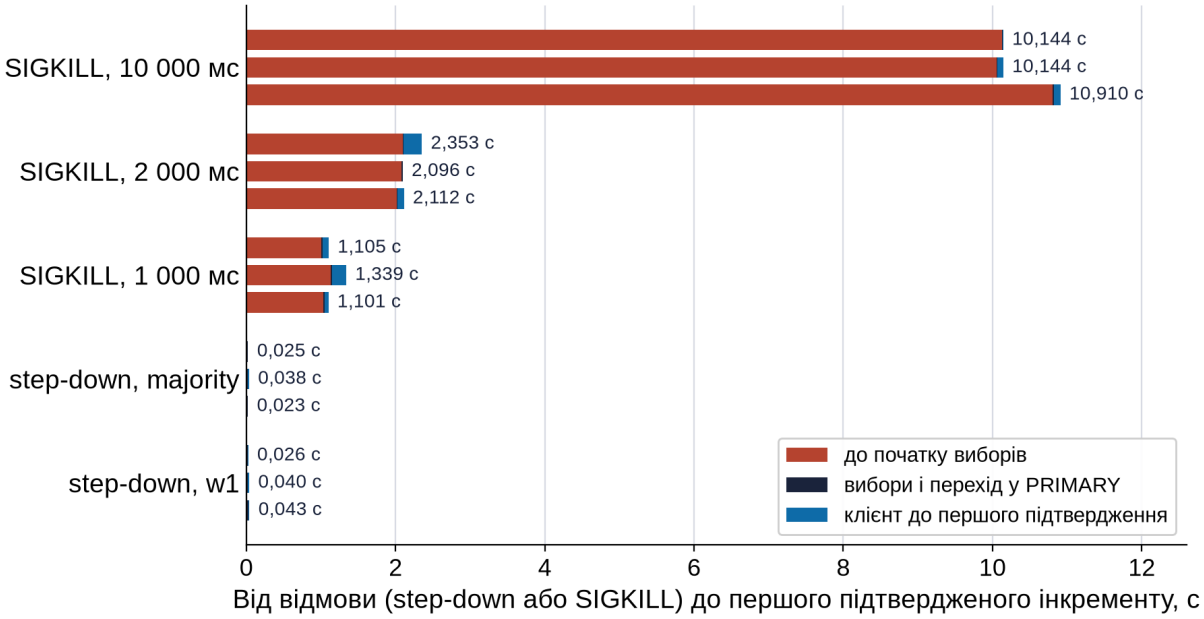



Рисунок 5.12. Пауза в записі після step-down і після SIGKILL при трьох значеннях `electionTimeoutMillis`: очікування до початку виборів, вибори і перехід у PRIMARY, очікування клієнта; кожен прогін окремо

Ціну короткого таймауту показує primary, що завис, а не впав: такий вузол не закриває з'єднань. Це змодельовано командами `pause-node` і `resume-node` без навантаження (`task4x-failover-pause.csv`).

Таблиця 5.14. Заморожений primary (SIGSTOP на 1,5 або 3 с) при різних `electionTimeoutMillis`: кількість заморожувань з виборами з трьох

| Таймаут, мс | Пауза 1,5 с | Пауза 3 с | 60 с без пауз |
|---|---|---|---|
| 10 000 | 0 з 3 | 0 з 3 | виборів немає |
| 2 000 | 0 з 3 | 3 з 3 | виборів немає |
| 1 000 | 3 з 3 | 3 з 3, у двох по двоє виборів | виборів немає |

У всіх 9 заморожуваннях з виборами новий primary дозволив запис лише через 0,002…0,015 с після того, як старий розморозився і склав роль: на 1,539…1,545 с після SIGSTOP при паузі 1,5 с і на 3,031…3,047 с при паузі 3 с. Без виборів запис відновлюється в момент розморожування. Отже, для коротких зависань короткий таймаут не скоротив перерву в записі, а лише змінив term, а з `w: 1` зміна term відкочує записи, яких не мав новий primary (підрозділ 5.1.1).

Причину видно з повних журналів перевірки (`task4x-failover-pause-probe.txt`). Обраний вузол входить у `primary catch-up mode` і чекає heartbeat від інших вузлів, а заморожений вузол визнається недоступним лише через `heartbeatTimeoutSecs` (10 с). Коли `electionTimeoutMillis` не більший за `heartbeatIntervalMillis` (2 000 мс), новий primary

рівно через electionTimeoutMillis після обрання пише Can't see a majority of the set, relinquishing primary і складає роль. Звідси подвійні вибори при 1 000 мс. Перевірка з паузою 13 с при 2 000 мс дала троє виборів поспіль (term 4, 5 і 6). Запис став можливим через 11,08 с після SIGSTOP, а старий primary після SIGCONT пройшов ROLLBACK за 0,29 с, хоча клієнтських записів не було.

Вузол після replSetStepDown: 60 60 с не може стати primary [14]. Два step-down поспіль і SIGKILL третього primary при 2 000 мс дали 49,356 с від SIGKILL до першого підтвердження замість близько 2 с. Обидва вузли, що лишились, писали Not starting an election ... still waiting for stepdown period to end, і вибори почались через 49,103 с, коли закінчився період першого з них. Прогін тривав 143,84 с з 2 000 повторів і дав рівно 100 000. Уперше це проявилось випадково в пробному прогоні з паузою 23,49 с (task4x-failover-smoke-stepdown-period.txt), тому task4x-failover.py починає кожен прогін не раніше ніж через 62 с після попереднього step-down. Плановий step-down, наприклад під час поетапного оновлення вузлів, на хвилину забирає вузол із кандидатів, і відмова в цей час триває довше за electionTimeoutMillis.

# 6 Висновки

1. Replica set MongoDB 8.2.12 з трьох вузлів (P-S-S) розгорнуто окремими процесами на одній машині. Write concern визначає лише момент відповіді: w: 3 без тайм-ауту чекав повернення вузла 10,623 с, а з wtimeout: 5000 повернув WriteConcernTimeout через 5,049 с, проте документ лишився записаним на двох вузлах і читався з readConcern: "majority".
2. Штатна зупинка primary передала роль за 0,029 с (stepUpRequestSkipDryRun), раптова (SIGKILL) зупинила запис на 10,3…11,3 с, бо secondary чекають electionTimeoutMillis 10 с. В усіх випадках старий primary повернувся як SECONDARY з усіма даними, записаними без нього.
3. Без відмов findOneAndUpdate з $inc точний при обох write concern: 100 000 за 22,49 с з w: 1 (4 447,0 інкр./с) і за 101,52 с з w: "majority" (985,1 інкр./с). majority у 4,5 раза повільніший, бо кожен інкремент чекає журналу на диску двох вузлів.
4. З w: 1 раптова відмова primary втратила 4, 1 і 0 підтверджених інкрементів: primary підтверджував їх до реплікації і до запису в журнал, тому вони зникли без rollback. З w: "majority" і retryable writes результат рівно 100 000 у всіх трьох прогонах з відмовою; сервер упізнав 7…10 повторів як уже виконані. Без lsid і txnNumber той самий сценарій дав 100 010.
5. Як сховище веб-лічильника MongoDB з w: "majority" дав 748,6 запитів/с: у 2,26 раза швидше за PostgreSQL (330,6) і в 6,1 раза за файл з fsync (122,2), бо інкременти чекають диска паралельно, але в 15 разів повільніше за Hazelcast (11 259,7) і в 41,8 раза за пам’ять (31 256,0).
6. Rollback справді відбувається, коли підтверджені записи є лише в журналі старого pri-

mary: 5 вставок і 5 інкрементів з `{ w: 1, j: true }` після його повернення відкочено в rollback-файли (6 документів і 12 записів oplog) як після SIGSTOP, так і після SIGKILL. Записи з `w: "majority"` збереглися.

7. `readConcern: "local"` двічі повернув дані, які потім відкотили: на старому primary і на ньому ж як SECONDARY до початку rollback. `majority` такого не показав, але відсталий secondary і з ним повернув застаріле значення (110 замість 111). Останнє підтверджене значення гарантовано дав лише `linearizable` на primary.
8. Затримка інкремента з `majority` має p50 9,7 мс і p99 16,9 мс, з `w: 1` p50 0,41 мс і p99 20,1 мс. SIGKILL primary видно лише в максимумі (10,4 с у 10 інкрементів, що були в дорозі) і як 10,40 с без жодного підтвердження на часовій шкалі.
9. Вузьке місце `majority` у синхронізації журналу: 82 % затримки primary чекає write concern, а журнал кожного вузла синхронізується 0,98 с на кожну секунду. Заповнення файлів журналу нулями (`log=(zero_fill=true)`) прибирає unwritten extents ext4 і без зміни правил дає 1 520,0 інкр./с замість 997,6 (+52,4 %) у `mongo-bench` і 1 474,6 запитів/с замість 870,7 (+69,4 %) у веб-лічильнику. Лічильник з 10 документів додає ще 7,5 % з `majority` і 49,4 % з `w: 1`, але змінює модель даних.
10. `replSetStepDown` під навантаженням зупиняє запис на 0,023…0,043 с і не губить записів навіть з `w: 1`; без `reconnect()` у клієнті пауза доходить до 5…10 с. `electionTimeoutMillis` 2 000 і 1 000 мс скорочують простій після SIGKILL з 10,144 с до 2,112 і 1,105 с (медіани), але заморожування primary на 3 с (при 1 000 мс уже на 1,5 с) дає вибори без виграшу в доступності. Step-down на 60 с забирає вузол із кандидатів, і після двох step-down відмова третього primary тривала 49,4 с.

# 7 Джерела


1. MongoDB Manual. Replication [Електронний ресурс]. Режим доступу: `https://www.mongodb.com/docs/manual/replication/` (дата звернення: 02.11.2026).
2. MongoDB Manual. Three Member Replica Sets [Електронний ресурс]. Режим доступу: `https://www.mongodb.com/docs/manual/core/replica-set-architecture-three-members/` (дата звернення: 02.11.2026).
3. MongoDB Manual. Deploy a Replica Set for Testing and Development [Електронний ресурс]. Режим доступу: `https://www.mongodb.com/docs/manual/tutorial/deploy-replica-set-for-testing/` (дата звернення: 02.11.2026).
4. MongoDB Manual. Replica Set Elections [Електронний ресурс]. Режим доступу: `https://www.mongodb.com/docs/manual/core/replica-set-elections/` (дата звернення: 02.11.2026).
5. MongoDB Manual. db.collection.findOneAndUpdate() [Електронний ресурс]. Режим доступу: `https://www.mongodb.com/docs/manual/reference/method/db.collection.findOneAndUpdate/` (дата звернення: 02.11.2026).

6. MongoDB Manual. Write Concern [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/reference/write-concern/ (дата звернення: 02.11.2026).
7. MongoDB Manual. Read Concern “majority” [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/reference/read-concern-majority/ (дата звернення: 02.11.2026).
8. MongoDB Manual. Retryable Writes [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/core/retryable-writes/ (дата звернення: 02.11.2026).
9. MongoDB Manual. Journaling [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/core/journaling/ (дата звернення: 02.11.2026).
10. MongoDB Specifications. Retryable Writes [Електронний ресурс]. Режим доступу: https://github.com/mongodb/specifications/blob/master/source/retryable-writes/retryable-writes.md (дата звернення: 02.11.2026).
11. MongoKitten [Електронний ресурс]. Режим доступу: https://github.com/orlandos-nl/MongoKitten (дата звернення: 02.11.2026).
12. MongoDB Manual. Rollbacks During Replica Set Failover [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/core/replica-set-rollbacks/ (дата звернення: 02.11.2026).
13. MongoDB Manual. Replica Set Configuration [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/reference/replica-configuration/ (дата звернення: 10.11.2026).
14. MongoDB Manual. replSetStepDown [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/reference/command/replSetStepDown/ (дата звернення: 10.11.2026).
15. MongoDB Manual. Read Concern [Електронний ресурс]. Режим доступу: https://www.mongodb.com/docs/manual/reference/read-concern/ (дата звернення: 10.11.2026).
16. WiredTiger. wiredtiger_open: log configuration [Електронний ресурс]. Режим доступу: https://source.wiredtiger.com/develop/group__wt.html (дата звернення: 10.11.2026).
17. fallocate(2). Linux manual page [Електронний ресурс]. Режим доступу: https://man7.org/linux/man-pages/man2/fallocate.2.html (дата звернення: 10.11.2026).